
<Company Name>

<Project Name>
Software Requirements Specification
For <Project/System/Feature>

Version <1.0>

Revision History

Date	Version	Description	Author
30/07/2026	1.0	<details>	

Table of Contents

1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions, Acronyms, and Abbreviations	5
1.4 References	6
1.4.1 Standards	6
1.4.2 Project Documentation	7
1.5 Overview	7
2. Overall Description	7
2.1 Product Perspective	7
2.2 Product Functions	8
Ticket Submission and Intake	8
Automated Ticket Analysis	8
Personally Identifiable Information (PII) Protection	8
Intelligent Ticket Routing	8
Retrieval-Augmented Generation (RAG)	8
Specialist AI Response Generation	8
Response Validation	8
Human Review and Escalation	9
Knowledge Management	9
Administration	9
2.3 User Classes and Characteristics	9
2.3.1 Customer / End User	9
2.3.2 Customer Support Agent	9
2.3.3 System Administrator	9
2.3.4 Knowledge Base Administrator	10
2.3.5 AI Operations Engineer	10
2.4 Operating Environment	10
Client Environment	10
Frontend Environment	10
Backend Environment	10
AI Environment	10
Data Storage	11
Deployment Environment	11
2.5 Design and Implementation Constraints	11
Architectural Constraints	11
AI Constraints	11
Security Constraints	11
Deployment Constraints	11

2.6 Assumptions and Dependencies	11
Assumptions	11
Dependencies	12
3. Specific Requirements	12
3.1 Functionality	12
3.1.1 User Authentication and Authorization	12
3.1.2 Ticket Management	14
3.1.3 Ticket Classification	17
3.1.4 Personally Identifiable Information (PII) Detection and Redaction	20
3.1.5 Supervisor Agent Orchestration	22
3.1.6 Specialist AI Agents	25
3.1.7 Retrieval-Augmented Generation (RAG)	27
3.1.8 Response Validation	30
3.1.9 Human Review and Escalation	32
3.1.10 Knowledge Base Management	34
3.1.11 User Management	36
3.1.12 System Administration	38
3.1.13 Logging and Monitoring	40
3.1.14 Reporting and Analytics	42
3.1.15 AI Model Management	44
3.1.16 Notification Services	45
3.1.17 Configuration Management	46
3.1.18 System Health Management	48
3.2 Usability	49
3.2.1 UR-001 -User Interface Consistency	49
3.2.2 UR-002 -Ease of Learning	49
3.2.3 UR-003 -Efficient Task Completion	50
3.2.4 UR-004 -Responsive User Interface	50
3.2.5 UR-005 -Accessibility	50
3.2.6 UR-006 -Error Prevention and Recovery	50
3.2.7 UR-007 -User Feedback	51
3.2.8 UR-008 -Dashboard Usability	51
3.2.9 UR-009 -Cross-Platform Compatibility	51
3.2.10 UR-010 -User Documentation and Help	51
3.3 Reliability	52
3.3.1 REL-001 -System Availability	52
3.3.2 REL-002 -Fault Tolerance	52
3.3.3 REL-003 -Workflow State Recovery	52
3.3.4 REL-004 -Data Integrity	53
3.3.5 REL-005 -AI Workflow Reliability	53
3.3.6 REL-006 -External Dependency Reliability	53

3.3.7 REL-007 -Error Recovery	54
3.3.8 REL-008 -Backup and Restore	54
3.3.9 REL-009 -Logging Reliability	54
3.3.10 REL-010 -Service Health Monitoring	54
3.3.11 REL-011 -Controlled System Shutdown	55
3.3.12 REL-012 -Continuous Operational Stability	55
3.4 Performance and Security	55
3.4.1 Performance Requirements	56
3.4.2 Security Requirements	58
Description	60
3.5 Supportability	61
3.5.1 SUP-001 -Modular System Architecture	61
3.5.2 SUP-002 -Configuration Management	61
3.5.3 SUP-003 -Logging and Diagnostic Information	61
3.5.4 SUP-004 -Health Monitoring	62
3.5.5 SUP-005 -Error Diagnosis	62
3.5.6 SUP-006 -Deployment Support	62
3.5.7 SUP-007 -Maintainability	63
3.5.8 SUP-008 -AI Model Support	63
3.5.9 SUP-009 -Knowledge Base Maintenance	63
3.5.10 SUP-010 -Operational Documentation	63
3.5.11 SUP-011 -Backup and Recovery Support	64
3.5.12 SUP-012 -Extensibility	64
3.6 Design Constraints	64
3.6.1 DC-001 -Architectural Constraints	64
3.6.2 DC-002 -Technology Stack Constraints	65
3.6.3 DC-003 -AI Processing Constraints	65
3.6.4 DC-004 -Security Constraints	65
3.6.5 DC-005 -Data Management Constraints	66
3.6.6 DC-006 -Deployment Constraints	66
3.6.7 DC-007 -Interface Constraints	66
3.6.8 DC-008 -Coding and Development Standards	67
3.6.9 DC-009 -Operational Constraints	67
3.6.10 DC-010 -Scalability Constraints	67
3.6.11 DC-011 -Compliance Constraints	67
3.6.12 DC-012 -Third-Party Dependency Constraints	68
3.6.13 DC-013 -Database Constraints	68
3.6.14 DC-014 -Vector Database Constraints	68
3.6.15 DC-015 -API Design Constraints	68
3.6.16 DC-016 -AI Model Constraints	69
3.6.17 DC-017 -Retrieval-Augmented Generation Constraints	69

3.6.18 DC-018 -User Interface Constraints	69
3.6.19 DC-019 -Desktop Application Constraints	70
3.6.20 DC-020 -Authentication Constraints	70
3.6.21 DC-021 -Logging Constraints	70
3.6.22 DC-022 -Configuration Constraints	70
3.6.23 DC-023 -Development Constraints	71
Software development shall follow established engineering practices.	71
• Source code shall be maintained using Git.	71
• Pull request reviews shall be required before merging.	71
• Continuous Integration shall execute automated quality checks.	71
• Unit and integration tests shall be executed before deployment.	71
• Source code passes automated validation.	71
• Code reviews are completed before integration.	71
3.7 On-line User Documentation and Help System Requirements	71
3.8 Purchased Components	71
3.9 Interfaces	71
3.9.1 User Interfaces	71
3.9.2 Hardware Interfaces	77
3.9.3 Software Interfaces	81
3.9.4 Communications Interfaces	87
3.10 Database Requirements	93
3.11 Licensing, Legal, Copyright, and Other Notices	93
3.12 Applicable Standards	93
4. Supporting Information	93

Clario: Customer Support Ticketing	Version: 1.0
Software Requirements Specification	Date: <dd/mmm/yy>
<document identifier>	

Software Requirements Specification

1. Introduction

This Software Requirements Specification (SRS) defines the complete software requirements for Clario, a multi-agent customer support triage and response system. The document specifies the functional and non-functional requirements, external interfaces, system constraints, quality attributes, and acceptance criteria required for the successful implementation, deployment, testing, and maintenance of the system.

This SRS has been prepared following the principles of IEEE 29148:2018 and IEEE 830 Software Requirements Specification standards. It serves as the primary reference for software developers, AI engineers, testers, project supervisors, maintainers, and other stakeholders throughout the project lifecycle.

The requirements contained within this document are derived from the approved project proposal, system architecture, backend architecture, frontend architecture, implementation roadmap, AI agent architecture, Retrieval-Augmented Generation (RAG) design, deployment architecture, and other supporting technical design documents developed for the Clario project. Any assumptions made within this document are explicitly identified to distinguish them from confirmed system requirements.

1.1 Purpose

The purpose of this Software Requirements Specification is to provide a complete, precise, and implementation-ready description of the Clario system.

This document establishes a common understanding among all stakeholders regarding the expected behavior, capabilities, interfaces, operational constraints, and quality attributes of the system. It serves as the authoritative source of software requirements throughout the software development lifecycle.

Specifically, this SRS aims to:

- Define the complete functional behavior of the Clario platform.
- Specify measurable non-functional requirements including performance, security, reliability, scalability, maintainability, usability, and availability.
- Describe the interactions between users, AI agents, external services, databases, and supporting infrastructure.
- Establish requirements for AI-driven ticket classification, routing, Retrieval-Augmented Generation (RAG), response validation, human escalation, and knowledge management.
- Provide acceptance criteria against which the final implementation can be verified and validated.
- Support software implementation, testing, deployment, future maintenance, and system evolution.
- Act as the contractual reference document for project implementation and future enhancements.

This document does not describe implementation details unless they directly influence externally observable system behavior or are required to satisfy system constraints.

1.2 Scope

Clario is an AI-powered customer support triage and response platform designed to automate the initial stages of customer support while maintaining human oversight for complex, high-risk, or low-confidence situations.

The system employs a multi-agent architecture orchestrated using LangGraph to coordinate specialized AI agents responsible for ticket analysis, domain-specific reasoning, Retrieval-Augmented Generation (RAG), response validation, and escalation.

The primary objectives of Clario are to:

- Automatically analyze incoming customer support tickets.
- Identify ticket category, urgency, priority, and customer sentiment.
- Route tickets to the appropriate specialist AI agent(s).
- Retrieve relevant organizational knowledge using Retrieval-Augmented Generation.
- Generate grounded, context-aware response drafts.
- Validate responses against organizational policies before presentation.

- Escalate unsupported or high-risk cases to human support personnel.
- Maintain an auditable history of AI reasoning and system decisions.

The system is intended primarily for educational institutions and customer support environments requiring intelligent ticket processing. The initial deployment target is integration with the Rysera STEM Learning Management System as a pilot environment; however, the architecture is designed to remain sufficiently modular for deployment in other customer support domains with minimal configuration changes.

The scope of the software includes:

- AI-assisted ticket triage
- Multi-agent orchestration
- Retrieval-Augmented Generation (RAG)
- Knowledge base management
- Human-in-the-loop review workflows
- Authentication and authorization
- Administrative configuration
- Logging and monitoring
- Deployment using containerized services

The following capabilities are outside the scope of the current release:

- Full CRM platform integration
- Enterprise-scale distributed deployment
- Autonomous customer communication without human oversight
- Voice-based customer support
- Advanced multilingual conversational support
- Continuous online model retraining
- Multi-provider enterprise authentication solutions beyond the defined architecture

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
AI	Artificial Intelligence
Agent	An autonomous software component responsible for performing specialized tasks within the orchestration workflow.
API	Application Programming Interface
Authentication	The process of verifying the identity of a user or service.
Authorization	The process of determining access permissions after authentication.
ChromaDB	Vector database used for storing document embeddings supporting Retrieval-Augmented Generation.
Classification Model	Fine-tuned language model responsible for predicting ticket category, priority, and sentiment.
Clario	The multi-agent customer support triage and response platform described in this document.

Docker	Containerization platform used to package and deploy software services.
FastAPI	Python web framework used to implement backend APIs.
Fine-Tuned LLM	A parameter-efficient adapted language model used for ticket classification tasks.
Human Handoff	Workflow in which responsibility for resolving a ticket is transferred from AI agents to a human support operator.
JWT	JSON Web Token used for stateless authentication.
Knowledge Base (KB)	Collection of approved support documents used by Retrieval-Augmented Generation.
LangGraph	Framework used to orchestrate multiple AI agents through explicit state-based workflows.
LLM	Large Language Model
LoRA	Low-Rank Adaptation method for efficient fine-tuning of language models.
PostgreSQL	Relational database management system used for application data storage.
RAG	Retrieval-Augmented Generation, combining retrieved knowledge with language model generation to produce grounded responses.
Supervisor Agent	Primary orchestration component responsible for coordinating the overall workflow.
Ticket	A customer support request submitted through the system.
Validation Agent	Component responsible for verifying response quality and policy compliance before delivery.
Vector Store	Database containing semantic document embeddings used during information retrieval.

1.4 References

The requirements defined within this document are based on the following project documentation and technical standards:

1.4.1 Standards

- IEEE Std 29148-2018 -Systems and Software Engineering -Life Cycle Processes -Requirements Engineering
- IEEE Std 830 -Software Requirements Specification
- ISO/IEC/IEEE 12207 -Software Life Cycle Processes

1.4.2 Project Documentation

1.5 Overview

The remainder of this Software Requirements Specification is organized as follows:

- **Section 2 – Overall Description** provides a high-level overview of the Clario system, including product perspective, user classes, operating environment, assumptions, dependencies, design constraints, and major product functions.
- **Section 3 – Specific Requirements** defines the complete functional and non-functional requirements of the system, including detailed system features, interface specifications, database requirements, security requirements, quality

attributes, operational constraints, logging, monitoring, error handling, compliance requirements, and measurable acceptance criteria.

- **Section 4 – Supporting Information** contains supplementary material including traceability information, appendices, supporting diagrams, assumptions, reference tables, and additional implementation guidance where applicable.

Together, these sections provide a complete specification for designing, implementing, testing, deploying, and maintaining the Clario platform while ensuring consistency with the approved project architecture and supporting technical documentation.

2. Overall Description

This section provides a high-level description of the Clario system, its intended purpose, operating context, primary stakeholders, major functional capabilities, system environment, design constraints, assumptions, and dependencies. It establishes the overall vision of the system without specifying detailed implementation requirements, which are defined in Section 3.

Clario is designed as an AI-powered customer support triage and response platform that combines a multi-agent orchestration framework, Retrieval-Augmented Generation (RAG), and a fine-tuned language model to automate the initial stages of customer support while ensuring that high-risk or uncertain cases remain under human supervision. The system is intended to improve response quality, reduce manual workload, provide consistent customer support, and maintain transparency through auditable AI-assisted decision making. The architecture, workflow, and deployment strategy described in this section are derived from the approved project proposal and implementation documents.

2.1 Product Perspective

Clario is a standalone software platform that operates as a modular customer support orchestration system. Rather than functioning as a traditional ticket management application, it acts as an intelligent processing layer positioned between end users submitting support requests and human support personnel responsible for resolving them.

The system receives incoming customer support tickets through a web-based user interface or external API, performs automated ticket analysis, routes requests through specialized AI agents, validates generated responses, and either returns a grounded response draft or escalates the request to a human support representative when appropriate.

The architecture follows a service-oriented, containerized design composed of multiple independent software services communicating through well-defined APIs. Major system components include:

- Web-based frontend application
- Spring Boot API Gateway (Java 17/21)
- Python FastAPI ML Sidecar (LangGraph Orchestration)
- Fine-Tuned Classification Model Service
- Retrieval-Augmented Generation (RAG) subsystem
- PostgreSQL relational database
- Chroma vector database
- Knowledge Base management subsystem
- Logging and monitoring infrastructure

Each component has a clearly defined responsibility and can evolve independently while maintaining compatibility through stable interface contracts.

The initial deployment targets an academic customer support environment, with Rysera STEM Learning Management System identified as the primary pilot platform. However, the architecture intentionally avoids Rysera-specific business logic, allowing the platform to be adapted to other customer support environments with minimal configuration changes.

2.2 Product Functions

At a high level, Clario provides the following major capabilities:

Ticket Submission and Intake

- Accept customer support requests through the web application and REST APIs.
- Validate incoming requests before processing.
- Generate unique ticket identifiers.
- Record ticket metadata and processing history.

Automated Ticket Analysis

- Detect ticket category.
- Predict ticket priority.
- Identify customer sentiment.
- Estimate classification confidence.
- Support single-domain and multi-domain ticket identification.

Personally Identifiable Information (PII) Protection

- Detect sensitive information within ticket content.
- Mask personally identifiable information before external AI processing.
- Preserve redaction metadata for auditing.

Intelligent Ticket Routing

- Route tickets to one or more specialist AI agents.
- Support dynamic routing decisions based on ticket characteristics.
- Support rerouting when validation identifies an incorrect specialist assignment.

Retrieval-Augmented Generation (RAG)

- Retrieve relevant knowledge base documents.
- Rank retrieved documents by semantic relevance.
- Build contextual prompts using retrieved knowledge.
- Ensure generated responses remain grounded in approved documentation.

Specialist AI Response Generation

- Generate domain-specific response drafts.
- Support multiple specialist agents operating independently.
- Combine outputs when multiple specialists participate in ticket resolution.

Response Validation

- Verify policy compliance.
- Detect unsupported or hallucinated responses.
- Evaluate response quality.
- Trigger bounded reflection where appropriate.
- Escalate unresolved issues.

Human Review and Escalation

- Escalate urgent, unsupported, or low-confidence cases.
- Produce structured handoff packages.
- Preserve reasoning history for human reviewers.

Knowledge Management

- Maintain versioned knowledge base documents.
- Support vector indexing.
- Manage retrieval metadata.
- Track knowledge source ownership.

Administration

- Manage authentication and authorization.
- Configure system settings.
- Monitor application health.
- View logs and operational metrics.

2.3 User Classes and Characteristics

The system supports multiple user categories with different responsibilities and permissions.

2.3.1 Customer / End User

Customers submit support tickets through the web application or integrated systems.

Characteristics:

- Minimal technical knowledge required.
- No understanding of AI workflows expected.
- Primary interaction consists of submitting support requests and viewing responses.

Primary permissions:

- Submit tickets.
- View submitted ticket status.
- Receive AI-assisted or human-reviewed responses.

2.3.2 Customer Support Agent

Support agents review escalated tickets and finalize responses.

Characteristics:

- Familiar with organizational support policies.
- Moderate technical knowledge.
- Responsible for resolving complex customer issues.

Primary permissions:

- Review AI-generated drafts.
- Modify responses.
- Approve or reject recommendations.
- Close support tickets.

2.3.3 System Administrator

Administrators manage the operational environment.

Characteristics:

- High technical expertise.
- Responsible for infrastructure, monitoring, deployment, and maintenance.

Primary permissions:

- Configure application settings.
- Manage users.
- Monitor system health.
- Access operational logs.
- Manage deployments.

2.3.4 Knowledge Base Administrator

Responsible for maintaining organizational documentation.

Primary responsibilities:

- Publish approved knowledge articles.
- Update outdated documentation.
- Manage document versions.

- Trigger knowledge indexing.

2.3.5 AI Operations Engineer

Responsible for AI lifecycle management.

Primary responsibilities:

- Monitor AI services.
- Evaluate model performance.
- Manage embeddings and vector indexes.
- Maintain classification models.
- Configure retrieval thresholds.

2.4 Operating Environment

The Clario platform is designed to operate in a modern cloud-native or on-premises containerized environment.

Client Environment

- Modern desktop web browsers
- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Safari

Frontend Environment

- React
- TypeScript
- Next.js
- Tailwind CSS

Backend Environment

- Java 17/21
- Spring Boot 3.x
- Spring Data JPA (Hibernate)
- Python 3.11
- FastAPI & Uvicorn
- LangGraph

AI Environment

- Fine-Tuned Open-Weight Language Model
- External reasoning LLM provider
- Sentence Transformer embedding model

Data Storage

- PostgreSQL
- Chroma Vector Database

Deployment Environment

- Docker
- Docker Compose

The system architecture allows deployment on Linux-based servers, cloud virtual machines, or container orchestration platforms supporting Docker-compatible workloads.

2.5 Design and Implementation Constraints

The implementation of Clario shall comply with the following constraints.

Architectural Constraints

- Multi-agent orchestration shall be implemented using LangGraph.
- Backend APIs shall be implemented using FastAPI.
- The frontend shall use React and TypeScript.
- Persistent relational data shall be stored in PostgreSQL.
- Semantic document retrieval shall use ChromaDB.
- AI components shall communicate through well-defined service interfaces.

AI Constraints

- External reasoning models shall never receive unredacted customer information.
- Generated responses shall be grounded using approved knowledge sources.
- AI outputs shall be validated before presentation.
- Unsupported responses shall not be presented as verified information.

Security Constraints

- JWT authentication shall protect secured APIs.
- Role-based authorization shall restrict privileged operations.
- Secrets shall not be stored in source code.
- Sensitive data shall be encrypted where applicable.

Deployment Constraints

- Services shall be containerized.
- Independent deployment of services shall be supported.
- Configuration shall be environment-driven.

2.6 Assumptions and Dependencies

The following assumptions apply to the current release.

Assumptions

- Users have reliable internet connectivity.
- Approved knowledge base documents are available before production deployment.
- External AI services remain available when configured.
- Human support personnel are available for escalated tickets.
- Organizational support policies remain externally maintained.

Dependencies

The system depends upon:

- FastAPI
- LangGraph
- PostgreSQL
- ChromaDB
- Sentence Transformers
- Docker
- External reasoning LLM provider
- Fine-tuned classification model service
- Approved knowledge base documents

Failure of any external dependency may reduce functionality, but the system should degrade gracefully by escalating requests

or returning appropriate operational error messages rather than generating unsupported responses. This behavior aligns with the project's architecture and orchestration plans.

3. Specific Requirements

This section specifies the detailed functional and non-functional requirements for the Clario platform. Each functional requirement defines a specific capability the system shall provide and includes the actors involved, preconditions, execution flow, alternative flows, postconditions, implementation priority, and measurable acceptance criteria.

All requirements defined within this section are uniquely identified to support traceability throughout software development, testing, deployment, and future maintenance activities.

3.1 Functionality

The Clario platform shall provide the functional capabilities described in this section. These requirements collectively define the externally observable behavior of the system and establish the baseline for implementation, verification, and validation.

3.1.1 User Authentication and Authorization

FR-001 -User Authentication

Description

The system shall authenticate registered users before allowing access to protected resources and services.

Actors

- Customer
- Support Agent
- System Administrator
- Knowledge Base Administrator
- AI Operations Engineer

Preconditions

- The user has a registered account.
- Authentication services are available.
- User credentials have been provided.

Main Flow

1. User submits authentication credentials.
2. The system validates the credentials.
3. The system verifies account status.
4. Upon successful authentication, the system generates JWT access and refresh tokens.
5. The system returns the authenticated user profile and authorization information.

Alternative Flows

A1 – Invalid Credentials

- Authentication fails.
- Error message is returned.
- No tokens are generated.

A2 – Disabled Account

- Login request is rejected.
- User is informed that the account is unavailable.

A3 – Authentication Service Failure

- Authentication request fails.
- Appropriate error response is returned.
- Failure is logged.

Postconditions

- Authenticated session is established.
- JWT tokens are issued.
- Authentication event is recorded.

Priority

High

Acceptance Criteria

- Only valid users can authenticate.
- Invalid credentials never produce tokens.
- JWT tokens contain required claims.
- Authentication completes within the specified performance limits.
- Authentication attempts are logged.

FR-002 -Authorization

Description

The system shall authorize access to protected resources according to assigned user roles.

Actors

- All authenticated users

Preconditions

- User is authenticated.
- Access token is valid.

Main Flow

1. User requests a protected resource.
2. System validates JWT token.
3. User role is determined.
4. Authorization policy is evaluated.
5. Access is granted or denied.

Alternative Flows

A1 – Invalid Token

- Request is rejected.
- HTTP 401 Unauthorized is returned.

A2 – Insufficient Permissions

- Request is denied.
- HTTP 403 Forbidden is returned.

Postconditions

- Authorized resource is returned OR
- Access is denied.

Priority

High

Acceptance Criteria

- Unauthorized users cannot access protected APIs.
- Role restrictions are consistently enforced.
- Authorization decisions are logged.

FR-003 -Session Management**Description**

The system shall manage authenticated user sessions using JWT access and refresh tokens.

Actors

- Authenticated Users

Preconditions

- User has successfully authenticated.

Main Flow

1. System issues access and refresh tokens.
2. User accesses protected resources.
3. Expired access tokens are refreshed using a valid refresh token.
4. Session continues without requiring re-authentication.

Alternative Flows*A1 – Refresh Token Expired*

- Refresh request is rejected.
- User must authenticate again.

Postconditions

- Session remains valid or is terminated.

Priority

Medium

Acceptance Criteria

- Tokens expire according to configuration.
- Refresh mechanism functions correctly.
- Expired refresh tokens cannot be reused.

3.1.2 Ticket Management

FR-004 -Ticket Submission

Description

The system shall allow users to submit customer support tickets through the user interface or REST API.

Actors

- Customer

Preconditions

- User is authenticated (where required).
- Ticket submission service is available.

Main Flow

1. User enters ticket information.
2. User submits the request.
3. System validates mandatory fields.
4. System generates a unique ticket identifier.
5. Ticket is stored.
6. Processing workflow begins.

Alternative Flows

A1 – Missing Required Fields

- Validation fails.
- User is prompted to correct input.

A2 – Database Failure

- Ticket is not stored.
- User receives an error message.

Postconditions

- Ticket exists within the system.
- Processing pipeline begins.

Priority

High

Acceptance Criteria

- Tickets receive unique identifiers.
- Validation rejects incomplete requests.
- Submitted tickets appear in the ticket queue.
- Submission events are logged.

FR-005 -Ticket Validation

Description

The system shall validate incoming ticket data before initiating AI processing.

Actors

- Customer
- API Client

Preconditions

- Ticket has been submitted.

Main Flow

1. System validates request schema.
2. Mandatory fields are verified.
3. Field formats are checked.
4. Invalid characters or malformed payloads are rejected.
5. Valid tickets continue to processing.

Alternative Flows

AI – Invalid Payload

- Validation fails.
- Processing does not begin.

Postconditions

- Only valid tickets enter the AI pipeline.

Priority

High

Acceptance Criteria

- Invalid requests are rejected.
- Schema validation is deterministic.
- Validation failures include descriptive error messages.

FR-006 -Ticket Status Management

Description

The system shall maintain and update the lifecycle status of every support ticket.

Actors

- Customer Support Agent
- AI Agents
- System Administrator

Preconditions

- Ticket exists.

Main Flow

1. Ticket enters processing.
2. Status changes as workflow progresses.
3. Final resolution updates ticket status.
4. Complete history is retained.

Supported Statuses

- Submitted
- Processing
- Awaiting AI Response
- Awaiting Human Review
- In Progress
- Resolved
- Closed

Alternative Flows

AI – Processing Failure

- Ticket status changes to Processing Failed.
- Human intervention may be required.

Postconditions

- Ticket lifecycle accurately reflects processing progress.

Priority

High

Acceptance Criteria

- Status transitions follow defined workflow rules.
- Invalid transitions are rejected.
- Complete status history is preserved.

FR-007 -Ticket Retrieval

Description

The system shall allow authorized users to retrieve ticket details and processing history.

Actors

- Customer
- Support Agent
- Administrator

Preconditions

- Ticket exists.
- User has permission.

Main Flow

1. User requests ticket.
2. System verifies authorization.
3. Ticket information is retrieved.
4. Processing history is displayed.

Alternative Flows

AI – Ticket Not Found

- System returns an appropriate error.

A2 – Access Denied

- Unauthorized access is rejected.

Postconditions

- Requested ticket information is returned.

Priority

Medium

Acceptance Criteria

- Authorized users can retrieve tickets.
- Unauthorized users cannot access restricted tickets.
- Retrieval operations are logged.

3.1.3 Ticket Classification

The Ticket Classification module is responsible for automatically analyzing newly submitted support tickets to determine their category, priority, urgency, sentiment, and confidence score before routing them through the multi-agent workflow.

FR-008 -Automatic Ticket Classification

Description

The system shall automatically classify every newly submitted support ticket using the fine-tuned classification model before initiating agent orchestration.

Actors

- Supervisor Agent
- Classification Service

Preconditions

- Ticket has passed validation.
- Classification model is available.

Main Flow

1. Supervisor Agent submits the ticket to the Classification Service.
2. Classification model analyzes the ticket.
3. Category is predicted.
4. Results are returned to the Supervisor Agent.
5. Classification metadata is stored.

Alternative Flows

A1 – Classification Model Unavailable

- Supervisor Agent records the failure.
- Ticket is escalated for manual processing.

A2 – Internal Classification Error

- Processing stops.

- Error is logged.
- Human review is initiated.

Postconditions

- Ticket category is assigned.
- Classification confidence is recorded.

Priority

High

Acceptance Criteria

- Every ticket is classified before routing.
- Classification metadata is stored.
- Model failures are recoverable.
- Classification events are logged.

FR-009 -Priority Prediction**Description**

The system shall predict the operational priority of each support ticket.

Actors

- Classification Service

Preconditions

- Ticket classification has begun.

Main Flow

1. Ticket content is analyzed.
2. Priority level is predicted.
3. Priority metadata is stored.
4. Priority is forwarded to the Supervisor Agent.

Supported Priority Levels

- Low
- Medium
- High
- Critical

Alternative Flows*A1 – Low Confidence Prediction*

- Ticket is marked for additional validation.

Postconditions

- Ticket priority is available.

Priority

High

Acceptance Criteria

- Every ticket receives a priority.
- Priority is stored.
- Low-confidence predictions are identifiable.

FR-010 -Sentiment Detection

Description

The system shall determine the overall customer sentiment expressed within the submitted ticket.

Actors

- Classification Service

Preconditions

- Ticket has been submitted.

Main Flow

1. Ticket text is analyzed.
2. Sentiment is predicted.
3. Sentiment is stored.

Supported Sentiments

- Positive
- Neutral
- Negative
- Highly Negative

Alternative Flows

A1 – Unable to Determine Sentiment

- Sentiment is recorded as Unknown.

Postconditions

- Sentiment metadata is available.

Priority

Medium

Acceptance Criteria

- Every processed ticket receives a sentiment label.
- Sentiment prediction is stored with classification metadata.

FR-011 -Confidence Score Generation

Description

The system shall generate confidence scores for all classification predictions.

Actors

- Classification Service

Preconditions

- Classification has completed.

Main Flow

1. Model computes prediction confidence.
2. Confidence score is attached to classification.
3. Supervisor Agent evaluates the confidence.

Alternative Flows

None.

Postconditions

- Confidence score is available.

Priority

High

Acceptance Criteria

- Confidence score accompanies every prediction.
- Confidence values remain within the configured range.

FR-012 -Multi-Label Classification**Description**

The system shall support assigning multiple categories to a single ticket when more than one business domain is identified.

Actors

- Classification Service
- Supervisor Agent

Preconditions

- Ticket content references multiple domains.

Main Flow

1. Multiple categories are predicted.
2. Confidence is calculated for each category.
3. Supervisor Agent receives all identified domains.

Alternative Flows

A1 – Single Domain Detected

- Only one category is assigned.

Postconditions

- One or more ticket categories exist.

Priority

Medium

Acceptance Criteria

- Multiple domains can be assigned.
- Category ordering reflects confidence.

3.1.4 Personally Identifiable Information (PII) Detection and Redaction

The PII Protection module safeguards sensitive customer information before any external AI model or service processes ticket content.

FR-013 -PII Detection**Description**

The system shall identify personally identifiable information contained within customer support tickets.

Actors

- PII Detection Service

Preconditions

- Ticket has passed validation.

Main Flow

1. Ticket content is scanned.
2. Sensitive entities are detected.
3. Detection metadata is stored.

Supported Entity Types

- Full names
- Email addresses
- Telephone numbers
- National identification numbers
- Physical addresses
- Student IDs
- Employee IDs
- Account numbers
- Customer identifiers

Alternative Flows

None.

Postconditions

- Sensitive entities are identified.

Priority

High

Acceptance Criteria

- Supported entity types are detected.
- Detection metadata is retained.

FR-014 -PII Redaction

Description

The system shall redact detected personally identifiable information before sending ticket content to external AI services.

Actors

- PII Redaction Service

Preconditions

- Sensitive information has been detected.

Main Flow

1. Sensitive values are replaced with placeholders.
2. Mapping information is securely preserved.
3. Redacted ticket is forwarded.

Example

Original:

My name is John Smith.

Processed:

My name is [PERSON_001].

Alternative Flows

AI – No PII Detected

- Original ticket proceeds unchanged.

Postconditions

- External AI receives only redacted content.

Priority

Critical

Acceptance Criteria

- External AI never receives unredacted PII.
- Original content remains securely stored.

FR-015 -PII Restoration

Description

The system shall restore original customer information when presenting responses to authorized users.

Actors

- Response Service

Preconditions

- Response generation has completed.

Main Flow

1. Placeholder mappings are retrieved.
2. Original values are restored.
3. Final response is presented.

Alternative Flows

A1 – Mapping Missing

- Restoration fails.
- Human review is initiated.

Postconditions

- Authorized users receive restored responses.\

Priority

High

Acceptance Criteria

- Restoration is accurate.
- Placeholder mappings remain protected.

3.1.5 Supervisor Agent Orchestration

The Supervisor Agent coordinates the execution of the complete multi-agent workflow and manages decision-making throughout the ticket lifecycle.

FR-016 -Workflow Initialization

Description

The Supervisor Agent shall initialize the processing workflow for every validated support ticket.

Actors

- Supervisor Agent

Preconditions

- Ticket validation completed.

Main Flow

1. Workflow state is created.
2. Ticket metadata is loaded.
3. Classification begins.
4. Processing state is initialized.

Alternative Flows

A1 – Initialization Failure

- Workflow terminates.
- Ticket is escalated.

Postconditions

- Workflow execution begins.

Priority

High

Acceptance Criteria

- Every valid ticket starts a workflow.
- Workflow state is persisted.

FR-017 -Dynamic Agent Routing**Description**

The Supervisor Agent shall dynamically determine which specialist AI agents should participate in ticket processing.

Actors

- Supervisor Agent

Preconditions

- Classification has completed.

Main Flow

1. Classification results are evaluated.
2. Required specialists are identified.
3. Execution sequence is determined.
4. Specialist agents are invoked.

Alternative Flows

A1 – No Matching Specialist

- Ticket is escalated.

Postconditions

- Appropriate specialists begin execution.

Priority

High

Acceptance Criteria

- Routing decisions use classification metadata.
- Multiple specialists may execute when necessary.

FR-018 -Workflow State Management**Description**

The Supervisor Agent shall maintain the complete execution state throughout the multi-agent workflow.

Actors

- Supervisor Agent

Preconditions

- Workflow is active.

Main Flow

1. Current workflow state is updated.
2. Agent outputs are recorded.
3. Execution progress is maintained.
4. State is persisted after each stage.

Alternative Flows

A1 – State Persistence Failure

- Workflow pauses.
- Recovery is attempted.

Postconditions

- Current workflow state is recoverable.

Priority

High

Acceptance Criteria

- Workflow state survives recoverable failures.
- Complete execution history is available.

FR-019 -Agent Coordination**Description**

The Supervisor Agent shall coordinate communication between participating AI agents.

Actors

- Supervisor Agent
- Specialist Agents

Preconditions

- Workflow is active.

Main Flow

1. Task assignments are issued.
2. Agent outputs are collected.
3. Intermediate context is shared.
4. Results are merged.

Alternative Flows*AI – Agent Timeout*

- Timeout is recorded.
- Alternative workflow path is executed.

Postconditions

- Coordinated workflow continues.

Priority

High

Acceptance Criteria

- Agent interactions remain synchronized.
- Communication failures are recoverable.

FR-020 -Workflow Completion**Description**

The Supervisor Agent shall determine when the processing workflow has successfully completed or requires escalation.

Actors

- Supervisor Agent

Preconditions

- All required processing stages have finished.

Main Flow

1. Processing results are evaluated.
2. Completion conditions are verified.
3. Ticket proceeds to response validation or human review.
4. Workflow is closed.

Alternative Flows*AI – Completion Conditions Not Met*

- Additional processing is initiated.

Postconditions

- Workflow reaches a terminal state.

Priority

High

Acceptance Criteria

- Every workflow ends in either:
 - Response generation,
 - Human escalation, or
 - Controlled failure.
- Terminal state is permanently recorded.

3.1.6 Specialist AI Agents

The Clario platform utilizes multiple specialized AI agents, each responsible for handling a specific business domain. These agents operate under the supervision of the Supervisor Agent and provide domain-specific reasoning, knowledge retrieval, and response generation.

FR-021 -Specialist Agent Selection

Description

The system shall invoke one or more specialist AI agents based on the ticket classification results produced by the Classification Service.

Actors

- Supervisor Agent
- Specialist AI Agents

Preconditions

- Ticket classification has completed successfully.
- At least one applicable specialist agent is available.

Main Flow

1. Supervisor Agent receives ticket classification results.
2. Required specialist agents are identified.
3. Selected specialist agents receive the ticket context.
4. Processing begins.

Alternative Flows

AI – Specialist Agent Unavailable

- Supervisor Agent records the failure.
- Ticket is reassigned or escalated.

Postconditions

- Appropriate specialist agents begin processing.

Priority

High

Acceptance Criteria

- Agent selection is based on classification results.
- Multiple agents may be selected when required.
- Selection decisions are recorded.

FR-022 -Specialist Reasoning

Description

Each specialist AI agent shall analyze assigned tickets using domain-specific reasoning and approved organizational knowledge.

Actors

- Specialist AI Agent

Preconditions

- Specialist agent has received the processing request.

Main Flow

1. Agent receives ticket context.
2. Relevant business rules are applied.
3. Retrieved knowledge is incorporated.
4. Domain-specific reasoning is performed.
5. Preliminary response is generated.

Alternative Flows

AI – Insufficient Context

- Agent requests additional information from the Supervisor Agent.

Postconditions

- Specialist reasoning results are available.

Priority

High

Acceptance Criteria

- Responses remain within the assigned business domain.
- Reasoning utilizes retrieved knowledge.
- Intermediate reasoning artifacts are stored.

FR-023 -Multi-Agent Collaboration

Description

The system shall support collaborative reasoning among multiple specialist agents when a ticket spans multiple business domains.

Actors

- Supervisor Agent
- Specialist AI Agents

Preconditions

- Multiple domains have been identified.

Main Flow

1. Supervisor Agent assigns work to multiple agents.
2. Each agent performs independent reasoning.
3. Results are collected.
4. Outputs are merged into a unified response.

Alternative Flows

AI – Conflicting Recommendations

- Response Validation Agent resolves inconsistencies.
- Human review may be requested.

Postconditions

- Consolidated reasoning output is produced.

Priority

High

Acceptance Criteria

- Multiple agents may participate in one workflow.
- Final output combines all required domain expertise.

FR-024 -Agent Failure Recovery

Description

The system shall recover from failures occurring within specialist AI agents without compromising workflow integrity.

Actors

- Supervisor Agent

Preconditions

- Specialist processing has begun.

Main Flow

1. Agent failure is detected.
2. Failure details are recorded.
3. Recovery strategy is selected.
4. Processing resumes or escalation occurs.

Alternative Flows

AI – Recovery Unsuccessful

- Ticket is escalated for human review.

Postconditions

- Workflow reaches a recoverable state or controlled termination.

Priority

High

Acceptance Criteria

- Agent failures do not corrupt workflow state.
- Recovery attempts are logged.

3.1.7 Retrieval-Augmented Generation (RAG)

The Retrieval-Augmented Generation subsystem retrieves approved organizational knowledge to ensure that AI-generated responses remain accurate, grounded, and aligned with organizational policies.

FR-025 -Knowledge Retrieval

Description

The system shall retrieve relevant knowledge base documents before generating customer responses.

Actors

- Retrieval Agent

Preconditions

- Ticket analysis has completed.
- Vector database is available.

Main Flow

1. Retrieval query is generated.
2. Semantic search is performed.
3. Relevant documents are ranked.
4. Top-ranked documents are returned.

Alternative Flows

AI – No Relevant Documents Found

- Retrieval confidence is marked as low.
- Workflow proceeds with appropriate safeguards.

Postconditions

- Retrieved documents are available for response generation.

Priority

High

Acceptance Criteria

- Retrieval utilizes semantic similarity.
- Document ranking is deterministic.
- Retrieval metadata is stored.

FR-026 -Context Construction**Description**

The system shall construct a contextual prompt using retrieved documents and ticket information.

Actors

- Context Builder

Preconditions

- Retrieval has completed.

Main Flow

1. Ticket information is combined with retrieved knowledge.
2. Duplicate information is removed.
3. Context is optimized within token limits.
4. Final prompt is generated.

Alternative Flows

A1 – Context Exceeds Token Budget

- Context is compressed according to prioritization rules.

Postconditions

- Optimized context is produced.

Priority

High

Acceptance Criteria

- Context includes only relevant information.
- Token budget is never exceeded.
- Context sources remain traceable.

FR-027 -Source Attribution**Description**

The system shall associate generated responses with the knowledge sources used during generation.

Actors

- Retrieval Agent
- Response Generator

Preconditions

- Response generation has completed.

Main Flow

1. Retrieved sources are tracked.
2. Source references are attached.
3. Metadata is stored.

Alternative Flows

None.

Postconditions

- Generated responses remain traceable.

Priority

Medium

Acceptance Criteria

- Every grounded response references its supporting knowledge.
- Source metadata is retained.

FR-028 -Knowledge Version Tracking**Description**

The system shall maintain version information for every knowledge document used during response generation.

Actors

- Knowledge Base Service

Preconditions

- Documents exist within the knowledge base.

Main Flow

1. Document version is identified.
2. Version metadata accompanies retrieval.
3. Processing history stores document versions.

Alternative Flows

None.

Postconditions

- Response generation remains reproducible.

Priority

Medium

Acceptance Criteria

- Knowledge versions are preserved.
- Historical workflows reference original document versions.

3.1.8 Response Validation

The Response Validation subsystem verifies AI-generated responses before they are presented to customers or support personnel.

FR-029 -Response Quality Validation**Description**

The system shall evaluate generated responses for completeness, relevance, and consistency.

Actors

- Validation Agent

Preconditions

- Response generation has completed.

Main Flow

1. Generated response is analyzed.
2. Validation rules are applied.
3. Quality score is calculated.
4. Validation result is returned.

Alternative Flows

AI – Validation Failure

- Response is rejected.
- Reflection or escalation begins.

Postconditions

- Validation outcome is available.

Priority

High

Acceptance Criteria

- All generated responses are validated.
- Validation results are stored.

FR-030 -Hallucination Detection

Description

The system shall identify unsupported statements within generated responses.

Actors

- Validation Agent

Preconditions

- Generated response exists.

Main Flow

1. Response is compared against retrieved knowledge.
2. Unsupported claims are identified.
3. Confidence is evaluated.
4. Response is accepted or rejected.

Alternative Flows

AI – Unsupported Content Detected

- Response enters reflection workflow.
- Human review may be required.

Postconditions

- Unsupported responses are prevented from delivery.

Priority

Critical

Acceptance Criteria

- Unsupported claims are detected.
- Validation decisions are recorded.

FR-031 -Reflection-Based Improvement

Description

The system shall support bounded reflection to improve low-quality AI responses before escalation.

Actors

- Validation Agent
- Specialist AI Agent

Preconditions

- Initial validation failed.

Main Flow

1. Validation identifies deficiencies.
2. Improvement instructions are generated.
3. Specialist agent revises the response.

4. Validation repeats.

Alternative Flows

AI – Maximum Reflection Attempts Reached

- Ticket proceeds to human review.

Postconditions

- Improved response or escalation is produced.

Priority

Medium

Acceptance Criteria

- Reflection attempts remain configurable.
- Infinite reflection loops cannot occur.

FR-032 -Final Response Approval

Description

The system shall determine whether a response is suitable for customer delivery or requires human intervention.

Actors

- Validation Agent
- Supervisor Agent

Preconditions

- Validation workflow has completed.

Main Flow

1. Validation results are reviewed.
2. Approval decision is made.
3. Approved responses proceed to delivery.
4. Rejected responses are escalated.

Alternative Flows

AI – Low Confidence Decision

- Human review is initiated.

Postconditions

- Workflow reaches an approved or escalated state.

Priority

High

Acceptance Criteria

- Only approved responses may be delivered.
- Approval decisions are permanently recorded.

3.1.9 Human Review and Escalation

The Human Review and Escalation module ensures that tickets requiring human judgment are transferred to appropriate support personnel while preserving the complete AI reasoning history to facilitate efficient resolution.

FR-033 -Automatic Escalation

Description

The system shall automatically escalate tickets to human support personnel when AI processing cannot confidently resolve the request.

Actors

- Supervisor Agent

- Support Agent

Preconditions

- AI processing has completed.
- Escalation criteria have been satisfied.

Main Flow

1. Supervisor Agent evaluates the workflow outcome.
2. Escalation conditions are verified.
3. Ticket is assigned to the appropriate support queue.
4. Support personnel are notified.
5. Ticket status is updated.

Alternative Flows*AI – No Available Support Agent*

- Ticket remains in the escalation queue.
- Retry according to system policy.

Postconditions

- Ticket is available for human review.

Priority

High

Acceptance Criteria

- Escalated tickets appear in the human review queue.
- Escalation reason is recorded.
- Ticket ownership is updated.

FR-034 -AI Reasoning History Preservation**Description**

The system shall preserve the complete AI reasoning history associated with every escalated ticket.

Actors

- Supervisor Agent
- Support Agent

Preconditions

- Ticket has been escalated.

Main Flow

1. AI workflow history is collected.
2. Classification results are included.
3. Retrieved knowledge references are attached.
4. Validation results are preserved.
5. Human reviewer receives the complete processing history.

Alternative Flows

None.

Postconditions

- Human reviewer has full visibility into AI processing.

Priority

High

Acceptance Criteria

- Processing history is complete.
- Reasoning artifacts remain immutable.
- Knowledge sources remain traceable.

FR-035 -Human Response Editing

Description

The system shall allow authorized support personnel to review, modify, approve, or replace AI-generated responses before delivery.

Actors

- Support Agent

Preconditions

- Ticket is assigned for review.

Main Flow

1. Support agent opens the ticket.
2. AI-generated response is displayed.
3. Agent edits or replaces the response.
4. Final response is approved.
5. Ticket proceeds to resolution.

Alternative Flows

AI – AI Response Rejected

- Agent creates a completely new response.

Postconditions

- Final approved response is stored.

Priority

High

Acceptance Criteria

- Human edits are preserved.
- Original AI response remains available for auditing.
- Approval history is recorded.

FR-036 -Ticket Resolution

Description

The system shall allow authorized support personnel to resolve and close customer support tickets.

Actors

- Support Agent

Preconditions

- Ticket has completed processing.

Main Flow

1. Final response is delivered.
2. Resolution status is selected.
3. Resolution notes are recorded.
4. Ticket status changes to Resolved or Closed.

Alternative Flows

AI – Additional Customer Information Required

- Ticket returns to In Progress.

Postconditions

- Ticket reaches a terminal lifecycle state.

Priority

High

Acceptance Criteria

- Closed tickets become read-only.
- Resolution metadata is retained.
- Ticket lifecycle remains auditable.

3.1.10 Knowledge Base Management

The Knowledge Base Management subsystem enables administrators to maintain organizational knowledge used by the Retrieval-Augmented Generation (RAG) pipeline.

FR-037 -Knowledge Document Management

Description

The system shall allow authorized users to create, update, archive, and delete knowledge base documents.

Actors

- Knowledge Base Administrator

Preconditions

- Administrator is authenticated.

Main Flow

1. Administrator uploads or edits a document.
2. Document metadata is validated.
3. Document is stored.
4. Version history is updated.

Alternative Flows

A1 – Invalid Document Format

- Upload is rejected.

Postconditions

- Knowledge repository is updated.

Priority

High

Acceptance Criteria

- Supported document formats are accepted.
- Document metadata is validated.
- Version history is maintained.

FR-038 -Automatic Knowledge Indexing

Description

The system shall automatically generate embeddings and update the vector database whenever approved knowledge documents change.

Actors

- Knowledge Indexing Service

Preconditions

- Knowledge document has been added or modified.

Main Flow

1. Updated document is detected.
2. Document is chunked.
3. Embeddings are generated.
4. Vector database is updated.
5. Retrieval index becomes available.

Alternative Flows

AI – Embedding Generation Failure

- Previous index remains active.
- Administrator is notified.

Postconditions

- Updated knowledge is searchable.

Priority

High

Acceptance Criteria

- New knowledge becomes searchable after indexing.
- Failed indexing does not corrupt existing indexes.

FR-039 -Knowledge Version Control

Description

The system shall maintain historical versions of knowledge documents.

Actors

- Knowledge Base Administrator

Preconditions

- Document already exists.

Main Flow

1. Administrator updates a document.
2. Previous version is archived.
3. New version becomes active.
4. Version metadata is stored.

Alternative Flows

None.

Postconditions

- Historical versions remain available.

Priority

Medium

Acceptance Criteria

- Previous versions cannot be lost.
- Active version is clearly identifiable.

FR-040 -Knowledge Search

Description

The system shall allow administrators to search and browse knowledge base documents.

Actors

- Knowledge Base Administrator

Preconditions

- Knowledge repository contains documents.

Main Flow

1. Search query is entered.
2. Matching documents are retrieved.
3. Results are displayed.

Alternative Flows

AI – No Results Found

- Empty result set is returned.

Postconditions

- Matching documents are displayed.

Priority

Medium

Acceptance Criteria

- Metadata search is supported.
- Search results are ordered by relevance.

3.1.11 User Management

The User Management module provides administrative capabilities for managing user accounts and access permissions.

FR-041 -User Registration**Description**

The system shall allow administrators to create user accounts.

Actors

- System Administrator

Preconditions

- Administrator is authenticated.

Main Flow

1. Administrator enters user information.
2. Validation is performed.
3. User account is created.
4. Initial credentials are generated.

Alternative Flows**A1 – Duplicate Account**

- Registration is rejected.

Postconditions

- User account exists.

Priority

Medium

Acceptance Criteria

- Duplicate accounts are prevented.
- User information is validated.

FR-042 -Role Management**Description**

The system shall support role assignment and modification for authorized users.

Actors

- System Administrator

Preconditions

- User account exists.

Main Flow

1. Administrator selects a user.
2. Role assignment is modified.
3. Changes are validated.
4. Permissions are updated.

Alternative Flows

A1 – Invalid Role Assignment

- Change is rejected.

Postconditions

- Updated permissions become effective.

Priority

High

Acceptance Criteria

- Role changes take effect immediately.
- Permission updates are logged.

FR-043 -Account Status Management

Description

The system shall support activation, suspension, and deactivation of user accounts.

Actors

- System Administrator

Preconditions

- User exists.

Main Flow

1. Administrator selects account status.
2. Status change is validated.
3. Account status is updated.

Alternative Flows

None.

Postconditions

- Updated account status becomes active.

Priority

Medium

Acceptance Criteria

- Suspended users cannot authenticate.
- Status changes are auditable.

3.1.12 System Administration

The System Administration module provides configuration, monitoring, and operational management capabilities for the Clario platform.

FR-044 -System Configuration Management

Description

The system shall allow authorized administrators to manage configurable application settings.

Actors

- System Administrator

Preconditions

- Administrator is authenticated.

Main Flow

1. Administrator modifies configuration.
2. Configuration is validated.

3. Changes are stored.
4. Services apply updated configuration where appropriate.

Alternative Flows

AI – Invalid Configuration

- Changes are rejected.

Postconditions

- Configuration becomes active.

Priority

High

Acceptance Criteria

- Configuration validation prevents invalid values.
- Configuration changes are logged.

FR-045 -Operational Dashboard

Description

The system shall provide administrators with an operational dashboard displaying system status and AI workflow metrics.

Actors

- System Administrator
- AI Operations Engineer

Preconditions

- Administrator is authenticated.

Main Flow

1. Dashboard is requested.
2. Operational metrics are collected.
3. Health indicators are displayed.
4. Current system status is presented.

Alternative Flows

AI – Monitoring Service Unavailable

- Dashboard displays partial information.

Postconditions

- Current operational state is visible.

Priority

Medium

Acceptance Criteria

- Dashboard updates with current operational information.
- Critical failures are clearly highlighted.

FR-046 -Audit Log Management

Description

The system shall maintain a secure audit log of all administrative and security-sensitive operations.

Actors

- System Administrator

Preconditions

- Administrative operation occurs.

Main Flow

1. Administrative event occurs.
2. Event metadata is collected.

3. Audit record is written.
4. Logs become searchable.

Alternative Flows

AI – Logging Failure

- System generates an operational alert.

Postconditions

- Audit history is preserved.

Priority

High

Acceptance Criteria

- Audit records cannot be modified by ordinary users.
- Administrative actions remain traceable.
- Log integrity is preserved.

3.1.13 Logging and Monitoring

The Logging and Monitoring subsystem provides comprehensive observability into system operations, AI workflows, security events, and infrastructure health. It enables administrators and AI operations engineers to diagnose issues, monitor system performance, and maintain operational reliability.

FR-047 -System Event Logging

Description

The system shall record all significant application events generated during normal operation.

Actors

- System Components
- Logging Service

Preconditions

- Application services are operational.

Main Flow

1. A system event occurs.
2. Relevant event metadata is collected.
3. Event is formatted according to the logging standard.
4. Event is written to the logging system.

Alternative Flows

AI – Logging Service Unavailable

- Events are temporarily buffered.
- Buffered events are written when logging service becomes available.

Postconditions

- System event is permanently recorded.

Priority

High

Acceptance Criteria

- Application events are logged.
- Log entries include timestamps and severity levels.
- Logs remain searchable.

FR-048 -AI Workflow Logging

Description

The system shall record the execution history of every AI workflow.

Actors

- Supervisor Agent
- Logging Service

Preconditions

- AI workflow has started.

Main Flow

1. Workflow execution begins.
2. Each processing stage is recorded.
3. Agent decisions are logged.
4. Workflow completion is recorded.

Alternative Flows

None.

Postconditions

- Complete AI workflow history is available.

Priority

High

Acceptance Criteria

- Workflow stages are traceable.
- Agent execution order is preserved.
- Processing duration is recorded.

FR-049 -Security Event Logging

Description

The system shall record authentication, authorization, and other security-sensitive events.

Actors

- Authentication Service
- Authorization Service

Preconditions

- Security event occurs.

Main Flow

1. Security event is detected.
2. Event metadata is collected.
3. Event is written to the audit log.

Alternative Flows

None.

Postconditions

- Security event is preserved.

Priority

Critical

Acceptance Criteria

- Login attempts are recorded.
- Authorization failures are logged.
- Administrative actions remain auditable.

FR-050 -Operational Monitoring

Description

The system shall continuously monitor operational metrics for all major software services.

Actors

- Monitoring Service

Preconditions

- Monitoring services are enabled.

Main Flow

1. Performance metrics are collected.
2. Service health is evaluated.
3. Operational dashboard is updated.
4. Alerts are generated when thresholds are exceeded.

Alternative Flows

A1 – Monitoring Service Failure

- Monitoring interruption is logged.

Postconditions

- Current operational status is available.

Priority

High

Acceptance Criteria

- Service metrics are updated continuously.
- Threshold violations trigger alerts.
- Monitoring history is retained.

3.1.14 Reporting and Analytics

The Reporting and Analytics subsystem provides operational insights into ticket processing, AI performance, and organizational support activities.

FR-051 -Ticket Analytics

Description

The system shall generate analytical summaries of customer support tickets.

Actors

- System Administrator
- AI Operations Engineer

Preconditions

- Ticket data exists.

Main Flow

1. Analytics request is received.
2. Ticket data is aggregated.
3. Statistical summaries are calculated.
4. Report is displayed.

Alternative Flows

None.

Postconditions

- Analytical report is available.

Priority

Medium

Acceptance Criteria

- Ticket statistics are accurate.

- Reports include configurable date ranges.

FR-052 -AI Performance Reporting

Description

The system shall provide reports describing AI system performance.

Actors

- AI Operations Engineer

Preconditions

- AI workflow data exists.

Main Flow

1. AI execution metrics are collected.
2. Performance indicators are calculated.
3. Report is generated.

Alternative Flows

None.

Postconditions

- AI performance report is available.

Priority

Medium

Acceptance Criteria

- Reports include processing time.
- Reports include escalation statistics.
- Reports include validation outcomes.

FR-053 -Export Reports

Description

The system shall allow authorized users to export generated reports.

Actors

- System Administrator

Preconditions

- Report exists.

Main Flow

1. User selects export format.
2. Report is generated.
3. Download begins.

Supported Formats

- PDF
- CSV
- Excel

Alternative Flows

AI – Export Failure

- User receives an error message.

Postconditions

- Exported report is available.

Priority

Low

Acceptance Criteria

- Exported reports preserve report contents.
- File generation is logged.

3.1.15 *AI Model Management*

The AI Model Management subsystem enables administrators to manage AI models and supporting machine learning assets used throughout the platform.

FR-054 -Model Registration

Description

The system shall support registration of approved AI models for production use.

Actors

- AI Operations Engineer

Preconditions

- Authorized administrator is authenticated.

Main Flow

1. Model metadata is submitted.
2. Validation is performed.
3. Model is registered.
4. Availability status is updated.

Alternative Flows

AI – Duplicate Model

- Registration is rejected.

Postconditions

- Model becomes available for deployment.

Priority

Medium

Acceptance Criteria

- Registered models contain complete metadata.
- Duplicate registrations are prevented.

FR-055 -Model Version Management

Description

The system shall maintain version information for deployed AI models.

Actors

- AI Operations Engineer

Preconditions

- Multiple model versions exist.

Main Flow

1. Model version is selected.
2. Deployment status is updated.
3. Previous versions remain archived.

Alternative Flows

None.

Postconditions

- Active model version is clearly identified.

Priority

Medium

Acceptance Criteria

- Model versions remain traceable.
- Previous versions remain recoverable.

FR-056 -Model Health Monitoring

Description

The system shall monitor the operational availability of AI models.

Actors

- AI Operations Engineer

Preconditions

- Model is deployed.

Main Flow

1. Health checks are executed.
2. Response latency is measured.
3. Availability status is updated.

Alternative Flows

AI – Model Unavailable

- Alert is generated.
- Supervisor Agent initiates fallback processing where applicable.

Postconditions

- Model health status is available.

Priority

High

Acceptance Criteria

- Model failures are detected.
- Health history is retained.
- Operational alerts are generated.

3.1.16 Notification Services

The Notification Service informs users of significant workflow events and operational activities.

FR-057 -Workflow Notifications

Description

The system shall notify users when significant ticket events occur.

Actors

- Notification Service

Preconditions

- Ticket workflow event occurs.

Main Flow

1. Event is detected.
2. Notification content is generated.
3. Notification is delivered.

Supported Events

- Ticket received
- Ticket escalated
- Ticket resolved

- Human review assigned

Alternative Flows

A1 – Notification Delivery Failure

- Delivery is retried according to system policy.

Postconditions

- Notification delivery status is recorded.

Priority

Medium

Acceptance Criteria

- Notifications correspond to workflow events.
- Delivery status is tracked.

FR-058 -Administrative Alerts

Description

The system shall notify administrators of critical operational conditions.

Actors

- Monitoring Service

Preconditions

- Critical condition detected.

Main Flow

1. Critical event occurs.
2. Alert is generated.
3. Administrator is notified.

Alternative Flows

None.

Postconditions

- Alert remains visible until acknowledged.

Priority

High

Acceptance Criteria

- Critical failures trigger alerts.
- Alerts remain auditable.

3.1.17 Configuration Management

The Configuration Management subsystem manages application settings controlling platform behavior.

FR-059 -Runtime Configuration

Description

The system shall allow authorized administrators to modify configurable operational settings.

Actors

- System Administrator

Preconditions

- Administrator is authenticated.

Main Flow

1. Configuration value is modified.
2. Validation is performed.

3. Configuration is saved.
4. Updated configuration becomes effective according to deployment policy.

Alternative Flows

A1 – Invalid Configuration

- Update is rejected.

Postconditions

- Configuration repository is updated.

Priority

Medium

Acceptance Criteria

- Invalid configuration values are rejected.
- Configuration changes are logged.

FR-060 -Configuration Backup and Recovery

Description

The system shall support backup and restoration of configuration settings.

Actors

- System Administrator

Preconditions

- Configuration repository exists.

Main Flow

1. Backup is initiated.
2. Configuration snapshot is created.
3. Backup is stored securely.

For restoration:

1. Backup is selected.
2. Configuration is restored.
3. System verifies integrity.

Alternative Flows

A1 – Backup Corrupted

- Restoration is rejected.

Postconditions

- Configuration is recoverable.

Priority

Low

Acceptance Criteria

- Backup integrity is verified.
- Restoration recreates previous configuration.

3.1.18 System Health Management

The System Health Management subsystem continuously evaluates the operational readiness of the Clario platform.

FR-061 -Health Check Service

Description

The system shall expose health endpoints for all major software services.

Actors

- Monitoring Service

- System Administrator

Preconditions

- Service is operational.

Main Flow

1. Health endpoint is requested.
2. Internal service checks execute.
3. Current status is returned.

Alternative Flows*AI – Service Failure*

- Health status is reported as unhealthy.

Postconditions

- Current service health is available.

Priority

High

Acceptance Criteria

- Health endpoints accurately reflect service status.
- Response includes dependency health.

FR-062 -Dependency Health Monitoring**Description**

The system shall continuously evaluate the availability of external dependencies.

Actors

- Monitoring Service

Preconditions

- External dependencies are configured.

Main Flow

1. Dependency health is checked.
2. Results are recorded.
3. Operational status is updated.

Monitored Dependencies

- PostgreSQL
- ChromaDB
- AI Models
- External LLM Provider
- Embedding Service

Alternative Flows*AI – Dependency Failure*

- System enters degraded operating mode where applicable.
- Administrator is notified.

Postconditions

- Dependency status is continuously available.

Priority

High

Acceptance Criteria

- Dependency failures are detected promptly.
- Degraded operation is supported where possible.
- Health history is retained.

3.2 Usability

The Clario platform shall provide a user-friendly, intuitive, and accessible interface that enables users with varying levels of technical expertise to efficiently perform their tasks. The system shall minimize the learning curve for new users while providing sufficient functionality for experienced users through a consistent and responsive user experience.

The usability requirements defined in this section establish measurable criteria for user interaction, interface consistency, accessibility, and overall user satisfaction.

3.2.1 UR-001 -User Interface Consistency

Description

The system shall provide a consistent user interface across all application modules.

Requirements

- Navigation components shall maintain a consistent layout throughout the application.
- Common user interface elements shall exhibit consistent behavior.
- Colors, typography, icons, and spacing shall follow a unified design system.
- Interactive elements shall provide consistent visual feedback.

Acceptance Criteria

- Interface components conform to the approved design system.
- Users can navigate between modules without relearning interface behaviors.

3.2.2 UR-002 -Ease of Learning

Description

The system shall be designed to minimize the learning effort required for first-time users.

Requirements

- Primary workflows shall be self-explanatory.
- Frequently used functions shall be easily discoverable.
- Contextual guidance shall be available where necessary.
- Error messages shall provide clear corrective instructions.

Acceptance Criteria

- New users can complete basic ticket submission and review tasks with minimal guidance.
- Navigation labels and interface terminology are clear and understandable.

3.2.3 UR-003 -Efficient Task Completion

Description

The system shall enable users to complete common tasks with minimal interaction.

Requirements

- Frequently performed operations shall require the fewest practical number of user interactions.
- Search and filtering capabilities shall be available where large datasets are presented.
- Frequently accessed information shall remain easily accessible.

Acceptance Criteria

- Users can submit, review, and manage tickets efficiently without unnecessary navigation.
- Common administrative operations require a minimal number of interaction steps.

3.2.4 UR-004 -Responsive User Interface

Description

The system shall provide responsive user interactions during normal operation.

Requirements

- User actions shall receive immediate visual feedback.
- Loading indicators shall be displayed during long-running operations.
- Progress indicators shall be shown for asynchronous processing tasks.

Acceptance Criteria

- Interactive elements provide visible feedback upon user interaction.
- Long-running operations display appropriate loading or progress indicators.

3.2.5 UR-005 -Accessibility

Description

The system shall support accessibility principles to accommodate users with diverse abilities.

Requirements

- User interface elements shall be keyboard accessible.
- Text shall maintain sufficient contrast against background colors.
- Images and icons shall include alternative text where appropriate.
- Forms shall include descriptive labels and validation messages.
- The interface shall support screen readers for essential workflows where technically feasible.

Acceptance Criteria

- Core application functionality is accessible using keyboard navigation.
- Text and interface elements satisfy recognized accessibility contrast recommendations.
- Form validation messages are understandable and descriptive.

3.2.6 UR-006 -Error Prevention and Recovery

Description

The system shall assist users in preventing errors and recovering from incorrect operations.

Requirements

- Required input fields shall be clearly identified.
- Input validation shall occur before data submission whenever possible.
- Confirmation dialogs shall be presented before destructive operations.
- Validation messages shall explain how users can resolve identified issues.

Acceptance Criteria

- Invalid input is detected before processing whenever feasible.
- Users receive clear instructions for correcting errors.
- Destructive operations require explicit user confirmation.

3.2.7 UR-007 -User Feedback

Description

The system shall provide timely and informative feedback for all significant user actions.

Requirements

- Successful operations shall display confirmation messages.
- Failed operations shall present meaningful error messages.
- System notifications shall clearly indicate the outcome of user actions.
- AI workflow progress shall be visible during ticket processing.

Acceptance Criteria

- Users receive immediate confirmation following successful operations.
- Error messages clearly describe the problem and any recommended corrective action.

- Ticket processing status is visible throughout the workflow.

3.2.8 *UR-008 -Dashboard Usability*

Description

The system shall provide role-specific dashboards presenting information relevant to each user category.

Requirements

- Customers shall view submitted tickets and their current status.
- Support agents shall view assigned and escalated tickets.
- Knowledge Base Administrators shall access document management functions.
- AI Operations Engineers shall access AI performance and operational metrics.
- System Administrators shall access system health, configuration, and monitoring information.

Acceptance Criteria

- Dashboard content corresponds to the authenticated user's assigned role.
- Frequently used information is immediately visible upon login.

3.2.9 *UR-009 -Cross-Platform Compatibility*

Description

The system shall provide a consistent user experience across supported desktop web browsers.

Requirements

- Interface layout shall remain consistent across supported browsers.
- Core application functionality shall operate without browser-specific modifications.
- Responsive layouts shall adapt appropriately to common desktop display resolutions.

Acceptance Criteria

- Core functionality operates consistently on supported browsers.
- Interface rendering remains consistent across supported platforms.

3.2.10 *UR-010 -User Documentation and Help*

Description

The system shall provide sufficient documentation and guidance to assist users in operating the platform effectively.

Requirements

- User documentation shall describe major system features.
- Administrative documentation shall explain system configuration and management procedures.
- Context-sensitive help shall be available for complex workflows where appropriate.
- Frequently encountered issues shall be documented with recommended solutions.

Acceptance Criteria

- Documentation is available for all major user roles.
- Users can access guidance without requiring external technical assistance for routine operations.

3.3 **Reliability**

The Clario platform shall provide reliable and predictable operation under normal and abnormal conditions. The system shall maintain data integrity, preserve workflow state, recover gracefully from failures, and ensure continuous availability of critical services wherever possible. Reliability requirements defined in this section establish measurable criteria for fault tolerance, recoverability, service continuity, and operational stability.

3.3.1 *REL-001 -System Availability*

Description

The system shall maintain a high level of operational availability during normal business operation.

Requirements

- Core application services shall remain continuously available except during scheduled maintenance.
- Planned maintenance activities shall be communicated in advance whenever possible.
- Service interruptions shall be minimized through controlled deployment procedures.

Acceptance Criteria

- System availability shall be at least **99.5%** during normal operating periods, excluding scheduled maintenance.
- Service interruptions shall be logged and monitored.

3.3.2 *REL-002 -Fault Tolerance*

Description

The system shall continue operating safely when individual software components experience recoverable failures.

Requirements

- Failure of one internal component shall not cause complete system failure.
- Independent services shall fail independently where applicable.
- Failed requests shall return meaningful error responses rather than causing unexpected system termination.
- Internal failures shall be isolated whenever possible.

Acceptance Criteria

- Individual component failures do not terminate unrelated services.
- Failure events are detected and recorded.
- Users receive informative error messages instead of unhandled exceptions.

3.3.3 *REL-003 -Workflow State Recovery*

Description

The system shall preserve workflow execution state to support recovery from unexpected interruptions.

Requirements

- Workflow state shall be persisted after each major processing stage.
- Interrupted workflows shall be recoverable where technically feasible.
- Recovery operations shall preserve workflow integrity.

Acceptance Criteria

- Recoverable failures do not require complete workflow restart.
- Workflow state remains consistent following recovery.

3.3.4 *REL-004 -Data Integrity*

Description

The system shall preserve the integrity and consistency of all stored application data.

Requirements

- Database transactions shall maintain atomicity and consistency.
- Invalid or partially processed data shall not be committed.
- Concurrent operations shall not corrupt stored information.
- Data relationships shall remain valid throughout system operation.

Acceptance Criteria

- Database integrity constraints are enforced.
- Failed transactions are rolled back automatically.
- No inconsistent data is introduced during normal operation.

3.3.5 REL-005 -AI Workflow Reliability

Description

The AI processing pipeline shall execute reliably throughout the complete ticket lifecycle.

Requirements

- AI workflow execution shall follow the defined orchestration sequence.
- Individual agent failures shall be detected promptly.
- Workflow failures shall result in controlled escalation rather than undefined behavior.
- AI reasoning history shall remain available for auditing.

Acceptance Criteria

- Workflow execution remains deterministic for identical processing conditions.
- Failed AI workflows transition to predefined recovery or escalation states.
- Processing history remains complete.

3.3.6 REL-006 -External Dependency Reliability

Description

The system shall monitor and manage failures of external services and dependencies.

Requirements

- Availability of databases, vector stores, AI providers, and supporting services shall be continuously monitored.
- Temporary failures shall trigger retry mechanisms where appropriate.
- Persistent failures shall initiate fallback procedures or human escalation.
- Dependency failures shall not compromise stored application data.

Acceptance Criteria

- External dependency failures are detected automatically.
- Operational alerts are generated for persistent failures.
- Graceful degradation is supported whenever possible.

3.3.7 REL-007 -Error Recovery

Description

The system shall recover gracefully from recoverable operational errors.

Requirements

- Recoverable errors shall not terminate application execution.
- Failed operations shall return descriptive error information.
- Recovery attempts shall follow predefined operational policies.
- Recovery events shall be recorded in the audit logs.

Acceptance Criteria

- Recoverable failures do not require manual system restart.
- Error recovery attempts are logged.
- Users receive appropriate feedback following recovery attempts.

3.3.8 REL-008 -Backup and Restore

Description

The system shall support reliable backup and restoration of critical application data and configuration.

Requirements

- Configuration data shall be backed up periodically.
- Application databases shall support backup and restoration procedures.
- Restoration processes shall verify backup integrity before recovery.

- Backup operations shall not significantly interrupt normal system operation.

Acceptance Criteria

- Backup integrity is verified before restoration.
- Restored systems preserve application consistency.
- Backup failures generate administrative alerts.

3.3.9 *REL-009 -Logging Reliability*

Description

The system shall ensure that operational logs remain reliable and complete.

Requirements

- Critical system events shall always be recorded.
- Log entries shall include timestamps and severity levels.
- Log storage failures shall not interrupt application execution.
- Audit logs shall remain protected against unauthorized modification.

Acceptance Criteria

- Critical operational events are retained.
- Log records remain searchable.
- Audit logs preserve integrity throughout system operation.

3.3.10 *REL-010 -Service Health Monitoring*

Description

The system shall ensure that operational logs remain reliable and complete.

Requirements

- Critical system events shall always be recorded.
- Log entries shall include timestamps and severity levels.
- Log storage failures shall not interrupt application execution.
- Audit logs shall remain protected against unauthorized modification.

Acceptance Criteria

- Critical operational events are retained.
- Log records remain searchable.
- Audit logs preserve integrity throughout system operation.

3.3.11 *REL-011 -Controlled System Shutdown*

Description

The system shall perform controlled shutdown procedures to prevent data corruption and preserve workflow integrity.

Requirements

- Active transactions shall be completed or rolled back safely.
- Workflow state shall be persisted before shutdown.
- Database connections shall close gracefully.
- Running background tasks shall terminate in a controlled manner.

Acceptance Criteria

- System shutdown does not corrupt application data.
- Workflow state remains recoverable after restart.
- Shutdown events are recorded in the audit log.

3.3.12 REL-012 -Continuous Operational Stability

Description

The system shall maintain stable operation during prolonged continuous execution.

Requirements

- Long-running services shall manage memory and computational resources efficiently.
- Background services shall operate without progressive degradation.
- Resource utilization shall remain within acceptable operational limits.
- Continuous execution shall not introduce memory leaks or resource exhaustion under normal workloads.

Acceptance Criteria

- Core services operate continuously without unexpected termination.
- Resource utilization remains stable during extended operation.
- System responsiveness remains within defined performance requirements.

3.4 Performance and Security

The Clario platform shall provide efficient, secure, and reliable operation while processing customer support requests and AI-assisted workflows. The system shall meet defined performance objectives under expected workloads and protect organizational data against unauthorized access, disclosure, modification, and loss.

This section defines the measurable performance and security requirements applicable to all software components within the Clario platform.

3.4.1 Performance Requirements

PER-001 -API Response Time

Description

The system shall provide responsive API services during normal operating conditions.

Requirements

- Authentication requests shall be processed efficiently.
- Ticket management operations shall return responses within acceptable response times.
- Administrative operations shall remain responsive during normal workloads.
- API response times shall be continuously monitored.

Acceptance Criteria

- At least **95%** of API requests shall complete within **2 seconds** under normal operating conditions.
- API response time metrics shall be available through the monitoring dashboard.

PER-002 -AI Workflow Processing Time

Description

The system shall complete AI-assisted ticket processing within an acceptable time for typical customer support requests.

Requirements

- Classification, orchestration, retrieval, generation, and validation shall execute as a coordinated workflow.
- AI processing time shall be monitored for every ticket.
- Processing bottlenecks shall be identifiable through operational metrics.

Acceptance Criteria

- The average AI processing time for a standard support ticket shall not exceed **15 seconds**, excluding delays introduced by external AI providers.
- Workflow execution duration shall be recorded for every ticket.

PER-003 -Concurrent User Support

Description

The system shall support multiple users accessing the platform simultaneously without significant degradation in performance.

Requirements

- Authentication services shall support concurrent login requests.
- Ticket submission and retrieval shall remain responsive during concurrent usage.
- Administrative operations shall continue functioning during normal user activity.

Acceptance Criteria

- The system shall support at least **100 concurrent authenticated users** under the expected deployment environment.
- Average response times shall remain within defined limits under expected workloads.

PER-004 -Database Performance

Description

The database subsystem shall efficiently process application transactions.

Requirements

- Database queries shall be optimized using appropriate indexing.
- Frequently accessed data shall be retrieved efficiently.
- Long-running queries shall be monitored.

Acceptance Criteria

- Standard database queries shall complete within **500 milliseconds** under normal operating conditions.
- Query execution statistics shall be available for monitoring.

PER-005 -Knowledge Retrieval Performance

Description

The Retrieval-Augmented Generation (RAG) subsystem shall retrieve relevant knowledge efficiently.

Requirements

- Semantic document retrieval shall utilize vector similarity search.
- Retrieved documents shall be ranked before response generation.
- Retrieval latency shall be monitored.

Acceptance Criteria

- Knowledge retrieval shall complete within **2 seconds** for the majority of requests under normal operating conditions.
- Retrieval performance metrics shall be logged.

PER-006 -Resource Utilization

Description

The system shall utilize computing resources efficiently.

Requirements

- CPU, memory, and storage utilization shall be monitored continuously.
- Resource consumption shall remain within configured operational thresholds.
- Resource exhaustion shall generate administrative alerts.

Acceptance Criteria

- Resource utilization statistics shall be continuously available.
- Threshold violations shall trigger operational notifications.

PER-007 -Scalability

Description

The system shall support future growth without requiring major architectural modifications.

Requirements

- Application services shall support horizontal scaling where appropriate.
- AI processing components shall be independently deployable.
- Database services shall support future performance optimization.

Acceptance Criteria

- Individual services can be deployed independently.
- Additional application instances can be introduced without application redesign.

PER-008 -Monitoring and Performance Metrics

Description

The system shall continuously collect operational performance metrics.

Requirements

- Response times shall be measured.
- AI workflow durations shall be recorded.
- Resource utilization statistics shall be collected.
- Performance trends shall be available for analysis.

Acceptance Criteria

- Historical performance metrics are retained.
- Performance reports accurately reflect operational behavior.

3.4.2 Security Requirements

SEC-001 -User Authentication

Description

The system shall authenticate all users before granting access to protected resources.

Requirements

- Authentication shall require valid user credentials.
- JWT access and refresh tokens shall be used for session management.
- Authentication failures shall be logged.

Acceptance Criteria

- Unauthorized users cannot access protected resources.
- Invalid credentials never produce valid authentication tokens.

SEC-002 -Authorization and Access Control

Description

The system shall enforce role-based access control (RBAC) for all protected operations.

Requirements

- Permissions shall be assigned according to user roles.
- Administrative functions shall be restricted to authorized personnel.
- Access control decisions shall be enforced consistently.

Acceptance Criteria

- Users can access only resources permitted by their assigned roles.
- Unauthorized requests return appropriate security responses.

SEC-003 -Password Security

Description

The system shall securely manage user credentials.

Requirements

- Passwords shall never be stored in plain text.
- Passwords shall be hashed using an industry-standard cryptographic hashing algorithm.
- Password verification shall occur only against stored password hashes.

Acceptance Criteria

- Plain-text passwords are never stored.
- Password hashes cannot be reversed through normal application functionality.

SEC-004 -Data Encryption

Description

The system shall protect sensitive information during storage and transmission.

Requirements

- Communication between clients and servers shall use HTTPS.
- Sensitive credentials shall be encrypted during transmission.
- Configuration secrets shall not be stored within application source code.

Acceptance Criteria

- All production network communication uses TLS.
- Sensitive configuration values are securely managed.

SEC-005 -Personally Identifiable Information (PII) Protection

Description

The system shall protect customer personally identifiable information throughout the AI processing pipeline.

Requirements

- PII shall be detected before external AI processing.
- Sensitive information shall be redacted where required.
- Authorized users shall receive restored information only after successful processing.

Acceptance Criteria

- External AI providers never receive unredacted protected information.
- PII detection and restoration remain auditable.

SEC-006 -Audit Logging

Description

The system shall maintain secure audit records of security-sensitive operations.

Requirements

- Authentication attempts shall be logged.
- Authorization failures shall be recorded.
- Administrative operations shall generate audit records.
- Audit logs shall be protected from unauthorized modification.

Acceptance Criteria

- Security events remain permanently traceable.
- Audit logs preserve integrity throughout their retention period.

SEC-007 -Session Security

Description

The system shall securely manage authenticated user sessions.

Requirements

- Access tokens shall expire after a configurable period.
- Refresh tokens shall be validated before issuing new access tokens.
- Expired or invalid tokens shall be rejected.

Acceptance Criteria

- Expired tokens cannot access protected resources.
- Session management complies with configured security policies.

SEC-008 -Input Validation

Description

The system shall validate all externally supplied input before processing.

Requirements

- API requests shall undergo schema validation.
- Invalid or malformed requests shall be rejected.
- Input validation shall reduce the risk of common injection attacks.

Acceptance Criteria

- Invalid requests are rejected before business logic execution.
- Validation failures return descriptive error messages.

SEC-009 -Secure API Communication

Description

The system shall protect all API endpoints against unauthorized access.

Requirements

- Protected endpoints shall require authentication.
- Authorization shall be verified before request execution.
- API requests shall be logged for security monitoring.

Acceptance Criteria

- Protected APIs cannot be accessed without valid authentication.
- Security violations are recorded.

SEC-010 -Secure AI Processing

Description

The system shall ensure that AI-assisted processing complies with organizational security and privacy requirements.

Requirements

- AI-generated responses shall be validated before delivery.
- Retrieved knowledge shall originate only from approved organizational sources.
- Unsupported or hallucinated AI responses shall not be presented as verified information.
- AI workflow decisions shall remain traceable.

Acceptance Criteria

- All generated responses pass validation before customer delivery.
- AI reasoning history remains available for auditing.
- Knowledge sources are recorded for every grounded response.

SEC-011 -Security Monitoring

Description

The system shall continuously monitor security-related events.

Requirements

- Authentication failures shall be monitored.
- Repeated unauthorized access attempts shall be logged.
- Security anomalies shall generate administrative alerts.

Acceptance Criteria

- Critical security events generate notifications.
- Security monitoring information is available through administrative dashboards.

SEC-012 -Configuration Security

Description

The system shall protect application configuration and operational secrets.

Requirements

- Configuration files containing sensitive information shall be protected.
- API keys, database credentials, and cryptographic secrets shall be stored using secure configuration mechanisms.
- Access to sensitive configuration shall be restricted to authorized administrators.

Acceptance Criteria

- Sensitive configuration values are never exposed through application interfaces.
- Configuration access is restricted and auditable.

3.5 Supportability

The Clario platform shall be designed to facilitate efficient maintenance, monitoring, troubleshooting, configuration, and future enhancements throughout its operational lifecycle. The system shall provide sufficient operational visibility, diagnostic capabilities, and administrative tools to enable developers, system administrators, and AI operations engineers to effectively deploy, monitor, maintain, and support the platform.

3.5.1 SUP-001 -Modular System Architecture

Description

The system shall employ a modular architecture that enables individual software components to be developed, tested, deployed, and maintained independently.

Requirements

- Major system components shall have clearly defined responsibilities.
- Components shall communicate through well-defined interfaces.
- Changes to one component shall minimize the impact on other components.
- New modules shall be integrated without significant architectural changes.

Acceptance Criteria

- Individual services can be updated independently.
- Component interfaces remain stable and well documented.
- System modifications require minimal changes to unrelated modules.

3.5.2 SUP-002 -Configuration Management

Description

The system shall support centralized and environment-specific configuration management.

Requirements

- Operational parameters shall be configurable without modifying application source code.
- Configuration values shall be stored externally using secure configuration mechanisms.
- Separate configurations shall be supported for development, testing, and production environments.
- Configuration changes shall be documented and auditable.

Acceptance Criteria

- Environment-specific configurations can be applied without recompiling the application.
- Configuration changes are logged and validated.

3.5.3 *SUP-003 -Logging and Diagnostic Information*

Description

The system shall provide comprehensive logging to support operational monitoring and fault diagnosis.

Requirements

- Application logs shall record significant system events.
- AI workflow execution shall be traceable through diagnostic logs.
- Errors shall include sufficient contextual information for troubleshooting.
- Log severity levels shall be consistently applied.

Acceptance Criteria

- Operational issues can be diagnosed using application logs.
- Log records contain timestamps, component identifiers, and severity levels.
- Diagnostic information is available without exposing sensitive user data.

3.5.4 *SUP-004 -Health Monitoring*

Description

The system shall provide mechanisms for monitoring the operational health of all critical software services.

Requirements

- Health endpoints shall be available for major application services.
- Internal dependency status shall be monitored continuously.
- Service availability shall be visible through administrative interfaces.
- Health monitoring shall support automated operational alerts.

Acceptance Criteria

- Health endpoints accurately report service status.
- Failed services are detected automatically.
- Health monitoring supports proactive maintenance.

3.5.5 *SUP-005 -Error Diagnosis*

Description

The system shall facilitate efficient identification and diagnosis of operational failures.

Requirements

- Error responses shall include unique identifiers where appropriate.
- Internal exceptions shall be recorded with sufficient diagnostic information.
- Operational failures shall be categorized according to severity.
- Troubleshooting information shall remain accessible to authorized administrators.

Acceptance Criteria

- Administrators can identify the source of reported failures.
- Diagnostic information supports efficient root-cause analysis.
- Error records remain searchable.

3.5.6 *SUP-006 -Deployment Support*

Description

The system shall support repeatable and consistent deployment procedures.

Requirements

- Software components shall support containerized deployment.
- Deployment configuration shall remain environment independent.
- Application startup and shutdown procedures shall be documented.

- Deployment validation shall verify service readiness.

Acceptance Criteria

- Application deployment follows documented procedures.
- Services initialize successfully after deployment.
- Deployment failures are detectable through health checks.

3.5.7 SUP-007 -Maintainability

Description

The software shall be structured to facilitate future maintenance and enhancement activities.

Requirements

- Source code shall follow established coding standards.
- Components shall maintain clear separation of responsibilities.
- Public interfaces shall remain well documented.
- Technical documentation shall be maintained alongside implementation changes.

Acceptance Criteria

- Code reviews confirm compliance with project coding standards.
- System modifications can be implemented without extensive redesign.
- Documentation remains synchronized with implementation.

3.5.8 SUP-008 -AI Model Support

Description

The system shall support maintenance and operational management of AI models used throughout the platform.

Requirements

- AI model versions shall be tracked.
- Model metadata shall be maintained.
- Model availability shall be monitored continuously.
- Model replacement shall not require application redesign.

Acceptance Criteria

- Active model versions are identifiable.
- Model health information is continuously available.
- Updated models can be deployed with minimal operational disruption.

3.5.9 SUP-009 -Knowledge Base Maintenance

Description

The system shall support efficient maintenance of the organizational knowledge base used by the Retrieval-Augmented Generation (RAG) subsystem.

Requirements

- Knowledge documents shall support version control.
- Document updates shall trigger controlled indexing procedures.
- Historical document versions shall remain recoverable.
- Indexing failures shall not affect existing production indexes.

Acceptance Criteria

- Updated documents become searchable after successful indexing.
- Previous document versions remain available.
- Knowledge maintenance activities are recorded in audit logs.

3.5.10 SUP-010 -Operational Documentation

Description

The system shall provide sufficient technical documentation to support deployment, operation, maintenance, and

troubleshooting.

Requirements

- Administrator documentation shall describe system configuration procedures.
- Deployment documentation shall describe installation and startup procedures.
- API documentation shall remain synchronized with implemented interfaces.
- AI workflow documentation shall describe the responsibilities of each agent within the orchestration process.

Acceptance Criteria

- Technical documentation is available for all major system components.
- Operational procedures can be executed using the provided documentation.
- Documentation is updated whenever significant system changes occur.

3.5.11 SUP-011 -Backup and Recovery Support

Description

The system shall support administrative backup and recovery procedures for critical operational data.

Requirements

- Application configuration shall support backup and restoration.
- Operational databases shall support backup procedures.
- Recovery procedures shall preserve system consistency.
- Backup integrity shall be verified before restoration.

Acceptance Criteria

- Backup operations complete successfully.
- Recovery restores a consistent operational state.
- Recovery procedures are documented and repeatable.

3.5.12 SUP-012 -Extensibility

Description

The system shall support future functional expansion without requiring significant architectural redesign.

Requirements

- Additional specialist AI agents shall be integrated through the existing orchestration framework.
- New knowledge sources shall be incorporated into the Retrieval-Augmented Generation pipeline using existing indexing mechanisms.
- Additional APIs and user interface modules shall conform to the established architectural patterns.
- Future enhancements shall preserve compatibility with existing interfaces whenever feasible.

Acceptance Criteria

- New system capabilities can be integrated with minimal modification to existing components.
- Existing functionality remains unaffected by supported extensions.
- Architectural consistency is maintained as the platform evolves.

3.6 Design Constraints

The design and implementation of the Clario platform shall comply with the architectural, technological, operational, and organizational constraints defined in this section. These constraints establish mandatory requirements that guide the system architecture, technology stack, software design decisions, and deployment strategy. All software components shall conform to these constraints unless formally approved exceptions are documented.

3.6.1 DC-001 -Architectural Constraints

Description

The system shall conform to the approved modular, service-oriented architecture to ensure maintainability, scalability, and separation of concerns.

Constraints

- The application shall follow a layered architecture consisting of presentation, application, domain, infrastructure, and data layers.
- AI processing shall be implemented as independent services coordinated through a centralized orchestration workflow.
- Components shall communicate through well-defined APIs and service interfaces.
- Business logic shall remain independent of presentation and infrastructure concerns.
- System modules shall exhibit low coupling and high cohesion.

Acceptance Criteria

- Major software components conform to the approved architectural design.
- Component responsibilities remain clearly separated.
- Cross-layer dependencies follow the defined architectural rules.

3.6.2 DC-002 -Technology Stack Constraints

Description

The implementation shall use the approved technology stack defined by the project architecture.

Constraints

- The frontend application shall be implemented using **React** and **TypeScript**.
- The transactional backend and API gateway shall be implemented using **Java** and **Spring Boot**, while the ML orchestration sidecar shall be implemented **using Python and FastAPI**.
- Multi-agent orchestration shall be implemented using **LangGraph**.
- Relational data shall be stored in **PostgreSQL**.
- Vector embeddings shall be stored in **ChromaDB**.
- Semantic retrieval shall utilize approved embedding models compatible with the RAG architecture.

Acceptance Criteria

- Implemented components utilize the approved technologies.
- Unsupported technologies are not introduced without architectural approval.

3.6.3 DC-003 -AI Processing Constraints

Description

AI-assisted processing shall operate within defined architectural and operational boundaries.

Constraints

- All customer tickets shall pass through the defined AI workflow.
- Ticket classification shall precede specialist agent execution.
- AI-generated responses shall be validated before customer delivery.
- Retrieval-Augmented Generation (RAG) shall utilize only approved organizational knowledge.
- Human escalation shall occur whenever configured confidence thresholds are not satisfied.

Acceptance Criteria

- AI workflows follow the approved orchestration sequence.
- Generated responses are validated before delivery.
- Unsupported AI outputs are not presented as verified responses.

3.6.4 DC-004 -Security Constraints

Description

The system shall comply with organizational security requirements throughout implementation.

Constraints

- Authentication shall be implemented using JWT-based authentication.
- Authorization shall follow Role-Based Access Control (RBAC).
- Personally Identifiable Information (PII) shall be protected during AI processing.

- Secrets, credentials, and API keys shall not be hardcoded within application source code.
- Administrative operations shall be auditable.

Acceptance Criteria

- Protected resources require authentication.
- Sensitive information is securely managed.
- Security-sensitive operations generate audit records.

3.6.5 DC-005 -Data Management Constraints

Description

Persistent data shall be managed according to approved database design principles.

Constraints

- Application data shall be stored in PostgreSQL.
- Knowledge embeddings shall be stored in ChromaDB.
- Database schema modifications shall preserve referential integrity.
- Historical audit records shall remain immutable where required.
- Data access shall occur through approved application services.

Acceptance Criteria

- Database integrity constraints are enforced.
- Data consistency is maintained throughout system operation.
- Schema changes follow documented migration procedures.

3.6.6 DC-006 -Deployment Constraints

Description

The platform shall support standardized deployment and operational management.

Constraints

- All major services shall support containerized deployment using Docker.
- Environment-specific configuration shall be externalized.
- Application services shall support independent deployment where applicable.
- Deployment procedures shall remain consistent across development, testing, and production environments.

Acceptance Criteria

- Services deploy successfully using the approved deployment process.
- Configuration changes do not require application recompilation.

3.6.7 DC-007 -Interface Constraints

Description

Software interfaces shall conform to established API and integration standards.

Constraints

- Internal services shall communicate using RESTful APIs or approved service interfaces.
- API contracts shall remain versioned and documented.
- Request and response payloads shall use JSON unless otherwise specified.
- Interface changes shall preserve backward compatibility whenever feasible.

Acceptance Criteria

- API implementations conform to documented interface specifications.
- Breaking interface changes require version updates.

3.6.8 DC-008 -Coding and Development Standards

Description

Software implementation shall follow established engineering practices to promote maintainability and code quality.

Constraints

- Source code shall follow project coding standards.
- Consistent naming conventions shall be used throughout the codebase.
- Source code shall be version controlled.
- Static analysis and code reviews shall be performed before integration.
- Public interfaces shall be documented.

Acceptance Criteria

- Code reviews verify compliance with project standards.
- Code quality checks pass before release.

3.6.9 DC-009 -Operational Constraints

Description

The system shall operate within defined operational boundaries.

Constraints

- System configuration shall be managed by authorized administrators only.
- Health monitoring shall remain enabled in production environments.
- Logging shall remain enabled for operational and security events.
- Critical services shall support graceful startup and shutdown procedures.

Acceptance Criteria

- Operational controls remain available to administrators.
- Service health and logging are continuously maintained.

3.6.10 DC-010 -Scalability Constraints

Description

The software architecture shall support future expansion without requiring fundamental architectural redesign.

Constraints

- Additional specialist AI agents shall integrate through the existing orchestration framework.
- New knowledge repositories shall integrate with the existing RAG pipeline.
- Additional application modules shall conform to the established architectural patterns.
- Future enhancements shall maintain compatibility with existing interfaces where practical.

Acceptance Criteria

- New capabilities can be integrated with minimal impact on existing components.
- Architectural consistency is preserved as the system evolves.

3.6.11 DC-011 -Compliance Constraints

Description

The implementation shall comply with applicable software engineering standards, organizational policies, and project governance requirements.

Constraints

- The Software Requirements Specification shall conform to IEEE 29148 and IEEE 830 recommendations.
- Software development activities shall follow the approved project lifecycle.
- Security controls shall comply with organizational information security policies.
- System documentation shall remain synchronized with implemented functionality.

Acceptance Criteria

- Project deliverables comply with documented standards.
- Documentation accurately reflects the implemented system.

3.6.12 DC-012 -Third-Party Dependency Constraints

Description

The system shall manage external libraries, frameworks, and AI services in a controlled and maintainable manner.

Constraints

- Third-party frameworks shall be actively maintained and supported.
- External AI providers shall be accessed only through approved integration interfaces.
- Dependency versions shall be documented and managed using the project's dependency management tools.
- Updates to third-party dependencies shall be evaluated for compatibility before deployment.

Acceptance Criteria

- Third-party dependencies are version controlled and documented.
- Dependency updates do not introduce incompatibilities or compromise system stability.

3.6.13 DC-013 -Database Constraints**Description**

The system shall adhere to the approved database architecture to ensure data consistency, integrity, and efficient data management.

Constraints

- PostgreSQL shall serve as the primary relational database.
- All persistent business entities shall be managed using the approved database schema.
- Database schema modifications shall be implemented through controlled migration scripts.
- Referential integrity constraints shall be enforced.
- Database access shall occur exclusively through the application's data access layer.

Acceptance Criteria

- Database schema conforms to the approved ER model.
- All schema changes are version-controlled and reversible.

3.6.14 DC-014 -Vector Database Constraints**Description**

The Retrieval-Augmented Generation subsystem shall utilize the approved vector database implementation.

Constraints

- ChromaDB shall be used for storing vector embeddings.
- Embeddings shall be generated only using approved embedding models.
- Vector indexes shall remain synchronized with the knowledge repository.
- Embedding metadata shall preserve document traceability.

Acceptance Criteria

- Vector indexes accurately represent the active knowledge base.
- Retrieval results remain traceable to original documents.

3.6.15 DC-015 -API Design Constraints**Description**

All backend services shall conform to the approved API design standards.

Constraints

- REST principles shall be followed.
- JSON shall be the standard message format.
- APIs shall expose OpenAPI documentation.
- Standard HTTP status codes shall be used consistently.
- API versioning shall be implemented for externally exposed endpoints.

Acceptance Criteria

- API documentation is automatically generated.
- Endpoints conform to documented interface specifications.

3.6.16 DC-016 -AI Model Constraints

Description

AI model integration shall comply with the approved architecture.

Constraints

- Large Language Models shall only be accessed through the AI service abstraction layer.
- AI providers shall remain interchangeable without affecting business logic.
- Prompt templates shall be maintained separately from application code.
- AI model configuration shall remain externally configurable.

Acceptance Criteria

- AI providers can be replaced without major application redesign.
- Prompt templates are centrally managed.

3.6.17 DC-017 -Retrieval-Augmented Generation Constraints

Description

The Retrieval-Augmented Generation pipeline shall follow the approved knowledge retrieval architecture.

Constraints

- Responses requiring organizational knowledge shall use the RAG pipeline.
- Only approved knowledge repositories shall be queried.
- Retrieved documents shall be ranked before generation.
- Source attribution shall be maintained throughout processing.

Acceptance Criteria

- Generated responses reference approved knowledge sources.
- Retrieval pipeline remains consistent with the architecture specification.

3.6.18 DC-018 -User Interface Constraints

Description

The user interface shall comply with the approved frontend architecture and design system.

Constraints

- React shall be used for implementing the user interface.
- TypeScript shall be used throughout the frontend.
- Shared UI components shall be reused whenever possible.
- Application styling shall follow the approved design system.

Acceptance Criteria

- Interface components remain visually consistent.
- Duplicate UI implementations are minimized.

3.6.19 DC-019 -Desktop Application Constraints

Description

The web application using React (Next.js)

Constraints

- Main Process shall manage backend lifecycle.
- FastAPI shall execute as a managed background process.
- Renderer processes shall communicate only through approved IPC mechanisms.
- Backend services shall terminate gracefully when the desktop application exits.

Acceptance Criteria

- Backend lifecycle is managed automatically.
- Desktop shutdown leaves no orphaned backend processes.

3.6.20 DC-020 -Authentication Constraints

Description

Authentication mechanisms shall comply with the approved security architecture.

Constraints

- JWT access and refresh tokens shall be used.
- Passwords shall never be stored in plain text.
- Authentication shall remain stateless.
- Session expiration shall be configurable.

Acceptance Criteria

- Authentication complies with the approved security model.
- Unauthorized access is prevented.

3.6.21 DC-021 -Logging Constraints

Description

Logging implementation shall follow the approved observability architecture.

Constraints

- Structured logging shall be used.
- Log formats shall remain consistent across all services.
- Correlation identifiers shall accompany distributed requests.
- Sensitive information shall never appear in logs.

Acceptance Criteria

- Logs support end-to-end request tracing.
- Sensitive information is excluded from operational logs.

3.6.22 DC-022 -Configuration Constraints

Description

Application configuration shall remain externally managed.

Constraints

- Environment variables shall be used for deployment-specific settings.
- Secrets shall not be committed to source control.
- Configuration validation shall occur during application startup.
- Invalid configuration shall prevent application initialization.

Acceptance Criteria

- Application starts only with valid configuration.
- Configuration values remain external to source code.

3.6.23 DC-023 -Development Constraints

Description

Software development shall follow established engineering practices.

Constraints

- Source code shall be maintained using Git.
- Pull request reviews shall be required before merging.
- Continuous Integration shall execute automated quality checks.
- Unit and integration tests shall be executed before deployment.

Acceptance Criteria

- Source code passes automated validation.
- Code reviews are completed before integration.

3.7 On-line User Documentation and Help System Requirements

The system shall provide context-sensitive help tooltips within the React frontend, particularly within the Agent Review and Ticket Submission interfaces. An online user manual detailing the ticket triage workflow and RAG knowledge ingestion process shall be hosted alongside the application deployment.

3.8 Purchased Components

There are no purchased software components required for the core execution of the Clario platform. The system relies on open-source frameworks and commercially available LLM APIs (e.g., Google Gemini) which are billed based on usage rather than purchased licensing.

3.9 Interfaces

The Clario platform shall provide well-defined interfaces to support user interaction, software component integration, hardware compatibility, and communication between distributed services. These interfaces shall enable seamless interaction between users, the desktop application, backend services, AI components, databases, and external AI providers while ensuring reliability, security, and maintainability.

The interface requirements defined in this section establish the expected behavior, supported technologies, communication protocols, and operational constraints for each interface category.

3.9.1 User Interfaces

The Clario platform shall provide a modern, intuitive, and role-based graphical user interface (GUI) that enables users to efficiently perform customer support operations, AI-assisted workflows, knowledge management, and system administration. The user interface shall be implemented as a web application using React (Next.js), following a consistent design system and ensuring a seamless user experience across all application modules.

The interface shall adapt according to the authenticated user's assigned role, displaying only the functions and information authorized for that role. All interfaces shall provide immediate feedback, support efficient navigation, and maintain consistency in layout, terminology, icons, and interaction patterns.

General User Interface Requirements

The user interface shall:

- Provide a consistent visual design throughout the application.
- Support role-based navigation and access control.
- Present responsive layouts suitable for standard desktop resolutions.
- Display clear validation messages for user inputs.
- Provide loading indicators during long-running operations such as AI processing and document indexing.
- Display informative success, warning, and error notifications.
- Support keyboard navigation for common operations.
- Prevent accidental destructive operations through confirmation dialogs.
- Display user profile information and authentication status.
- Maintain a consistent color scheme, typography, icons, and spacing across all modules.

Authentication Interface

The Authentication Interface shall provide secure access to the application.

Required Components

- Username or Email input field
- Password input field
- Login button
- Forgot Password option
- Password visibility toggle

- Error message display area
- Loading indicator during authentication

Functions

- Authenticate registered users
- Validate user credentials
- Create authenticated user sessions
- Display authentication errors
- Redirect authenticated users to their dashboards

Dashboard Interface

The Dashboard shall provide users with an overview of system information relevant to their assigned role.

Required Components

- Navigation sidebar
- Application header
- User profile menu
- Notification center
- Dashboard summary cards
- Recent activity panel
- Quick action buttons
- AI workflow status indicators

Functions

- Display role-specific information
- Navigate to application modules
- View pending tasks
- Display operational summaries
- Access user profile and logout

Ticket Management Interface

The Ticket Management Interface shall enable users to create, monitor, and manage customer support tickets.

Required Components

- Ticket list
- Search bar
- Advanced filtering panel
- Sorting controls
- Ticket details panel
- Ticket creation form
- Status selector
- Priority selector
- Category selector
- Rich-text comment area
- File attachment section
- Activity history panel

Functions

- Submit new support tickets
- View ticket details
- Update ticket information
- Search and filter tickets
- Add comments and attachments

- Monitor ticket status
- View ticket history

AI Workflow Interface

The AI Workflow Interface shall provide visibility into the AI-assisted ticket processing pipeline.

Required Components

- Workflow progress indicator
- Ticket classification results
- Assigned specialist agent information
- Retrieved knowledge references
- AI-generated response panel
- Validation status
- Confidence score indicator
- Processing timeline
- Functions
 - Display workflow execution progress
 - Present AI-generated responses
 - Display supporting knowledge sources
 - Show validation outcomes
 - Display escalation status

Human Review Interface

The Human Review Interface shall support manual validation of AI-generated responses before customer delivery.

Required Components

- Customer ticket details
- Original customer message
- AI-generated response
- Supporting knowledge references
- Editable response editor
- Approval button
- Reject button
- Escalation button
- Resolution notes field
- Functions
 - Review AI-generated responses
 - Edit response content
 - Approve responses
 - Reject responses
 - Escalate tickets
 - Record review comments

Knowledge Base Management Interface

The Knowledge Base Management Interface shall enable authorized users to manage organizational knowledge used by the Retrieval-Augmented Generation (RAG) subsystem.

Required Components

- Document library
- Search bar
- Upload document button

- Document metadata editor
- Version history viewer
- Indexing status indicator
- Delete and archive controls

Functions

- Upload knowledge documents
- Update document metadata
- Delete or archive documents
- Search knowledge assets
- Monitor indexing progress
- View document versions

AI Model Management Interface

The AI Model Management Interface shall support operational management of AI models.

Required Components

- Registered model list
- Model version information
- Active model indicator
- Health status panel
- Configuration editor
- Deployment controls

Functions

- View registered AI models
- Activate model versions
- Monitor model health
- Configure AI providers
- Review model metadata

User Management Interface

The User Management Interface shall enable administrators to manage user accounts and permissions.

Required Components

- User list
- User details panel
- Role assignment controls
- Search and filter options
- Account status indicator
- Create, update, and deactivate controls

Functions

- Register users
- Update user information
- Assign user roles
- Activate or deactivate accounts
- Search registered users

System Administration Interface

The System Administration Interface shall provide centralized management of operational settings and system resources.

Required Components

- Configuration management panel

- System health dashboard
- Service status indicators
- Audit log viewer
- Monitoring dashboard
- Backup and recovery controls
- Application settings panel
- Functions
 - Configure application settings
 - Monitor service health
 - View audit logs
 - Manage system configuration
 - Perform backup and recovery operations

Notification Interface

The Notification Interface shall provide real-time visibility of system events and workflow updates.

Required Components

- Notification panel
- Notification history
- Alert badges
- Severity indicators
- Read and unread status
- Functions
 - Display workflow notifications
 - Present operational alerts
 - Show security notifications
 - Track notification history

Profile and Account Management Interface

The Profile Interface shall enable users to manage their personal account information.

Required Components

- User profile information
- Password change form
- Profile editing form
- Account preferences
- Session information
- Functions
 - View profile information
 - Update account details
 - Change password
 - Manage personal preferences

Error and Validation Interface

The system shall provide standardized mechanisms for displaying validation messages and operational errors.

Interface Characteristics

- Required fields shall be clearly identified.
- Validation shall occur before form submission where possible.
- Error messages shall clearly describe the identified issue.
- Success messages shall confirm completed operations.

- Warning messages shall identify actions requiring user attention.
- Critical system failures shall display reference identifiers for troubleshooting.

Navigation Structure

The application shall provide a consistent navigation structure including:

- Dashboard
- Tickets
- AI Processing
- Human Review
- Knowledge Base
- AI Models
- Notifications
- User Management (Administrator)
- System Administration (Administrator)
- Settings
- Help and Documentation
- User Profile
- Logout

Navigation options shall be displayed according to the authenticated user's assigned role and permissions.

User Interface Accessibility Requirements

The graphical user interface shall:

- Support keyboard navigation for primary workflows.
- Maintain adequate color contrast for readability.
- Provide descriptive labels for all form controls.
- Display meaningful validation and error messages.
- Maintain consistent terminology across all modules.
- Support scalable interface elements for different display resolutions.
- Avoid unnecessary user interactions by streamlining common workflows.

The user interface shall prioritize clarity, consistency, responsiveness, and usability to enable efficient operation of the Clario platform while minimizing user effort and reducing the likelihood of operational errors.

3.9.2 Hardware Interfaces

The Clario platform is designed as a cross-platform desktop application that operates on standard computing hardware without requiring specialized or proprietary hardware devices. The system follows a Hybrid Monolithic Architecture, where the application consists of a Spring Boot application, a Python FastAPI ML Sidecar, and a PostgreSQL database. These components may execute directly on the host machine during development or within Docker containers in production while sharing the same physical hardware resources.

The hardware interface requirements specified in this section define the minimum and recommended client hardware, storage requirements, processing capabilities, and hardware interactions necessary for reliable operation of the Clario platform.

Client Hardware Requirements

Description

The system shall operate on standard desktop and laptop computers without requiring dedicated hardware peripherals.

Minimum Hardware Requirements

Component	Minimum Requirement
Processor	64-bit Dual-Core Processor (Intel Core i3 / AMD Ryzen 3 or equivalent)
Memory (RAM)	8 GB
Storage	5 GB available disk space
Display Resolution	1366 × 768 pixels
Network	Broadband Internet connection
Graphics	Integrated Graphics supporting Web application

Recommended Hardware Requirements

Component	Recommended Requirement
Processor	Quad-Core 64-bit Processor (Intel Core i5/i7 or AMD Ryzen 5/7 or equivalent)
Memory (RAM)	16 GB or higher
Storage	SSD with at least 10 GB available space
Display Resolution	Full HD (1920 × 1080) or higher
Network	Stable broadband Internet connection
Graphics	Hardware acceleration supported by modern GPU

Acceptance Criteria

- The application installs and operates successfully on hardware meeting the minimum specifications.
- All core functionalities remain available on the supported hardware configurations.

Server Hardware Requirements

Description

The backend services shall operate on commodity server hardware capable of hosting the Spring Boot application, PostgreSQL database, and Python ML Sidecar.

Minimum Server Requirements

Component	Minimum Requirement
Processor	4 CPU Cores

Memory (RAM)	16 GB
Storage	100 GB SSD
Network	Gigabit Ethernet Connectivity

Recommended Server Requirements

Component	Recommended Requirement
Processor	8 CPU Cores or higher
Memory (RAM)	32 GB or higher
Storage	SSD with sufficient capacity for application data and knowledge base
Network	High-speed network with low latency

Acceptance Criteria

- Backend services operate reliably under expected workloads.
- Hardware resources support concurrent user access and AI-assisted processing.

AI Processing Hardware

Description

The Python ML Sidecar shall support execution on both CPU-only systems and GPU-accelerated systems depending on the selected AI models and deployment environment.

Requirements

- The AI processing service shall support execution using standard CPU resources.
- GPU acceleration may be utilized when compatible hardware is available.
- AI processing components shall operate independently of the transactional backend.
- Hardware-specific optimizations shall not affect application functionality.

This architecture allows the Spring Boot application to continue serving transactional operations even if the AI sidecar experiences hardware or runtime failures, supporting the project's Hybrid Monolithic deployment model.

Acceptance Criteria

- AI services execute correctly on supported hardware configurations.
- The transactional application remains operational when AI hardware resources are unavailable.

Storage Interfaces

Description

The system shall utilize persistent storage devices for application data, configuration files, uploaded documents, and knowledge repositories.

Requirements

The storage subsystem shall support:

- PostgreSQL relational database files.
- ChromaDB vector database storage.
- Uploaded knowledge base documents.
- Application logs and audit records.

- Configuration files.
- Backup and recovery data.

Acceptance Criteria

- Persistent storage maintains data integrity across system restarts.
- Storage devices provide sufficient capacity for operational data.

File System Interface

Description

The application shall interact with the host operating system's file system for document management and application configuration.

Requirements

The system shall support:

- Uploading knowledge base documents.
- Reading configuration files.
- Writing application logs.
- Exporting reports.
- Importing administrative data.
- Creating backup archives.

The application shall validate all uploaded files before processing and prevent unauthorized access to protected file locations.

Acceptance Criteria

- File operations execute successfully for authorized users.
- Invalid or unsupported files are rejected with appropriate error messages.

Network Interface

Description

The system shall utilize standard network interfaces for communication with internal and external services.

Requirements

The hardware shall support:

- TCP/IP networking.
- Internet connectivity for cloud-based AI providers.
- Localhost communication between the Spring Boot application and the Python FastAPI ML Sidecar.
- Docker bridge networking when deployed using containers.

The communication between the Spring Boot backend and the FastAPI sidecar shall occur over the local host network (or Docker internal network), minimizing latency while maintaining logical separation between transactional and AI processing components.

Acceptance Criteria

- Internal services communicate reliably over supported network interfaces.
- External AI services are reachable when Internet connectivity is available.

Display Interface

Description

The system shall provide graphical output through standard desktop display hardware.

Requirements

The application shall:

- Support common desktop monitor resolutions.
- Scale user interface elements appropriately for different display sizes.

- Maintain readability across supported display resolutions.

Acceptance Criteria

- User interface components render correctly on supported display devices.
- Text and graphical elements remain clear and legible.

Input Device Interface

Description

The application shall support standard desktop input devices.

Requirements

The system shall support:

- Keyboard input.
- Mouse input.
- Touchpad input.
- Standard clipboard operations (copy, cut, and paste).

No specialized hardware input devices shall be required.

Acceptance Criteria

- All major application functions are accessible using standard desktop input devices.
- Input devices operate consistently throughout the application.

Containerized Deployment Environment

Description

The system shall support execution within a containerized environment to ensure platform consistency and dependency isolation.

Requirements

The deployment environment shall support containerization of:

- Spring Boot Application
- Python FastAPI ML Sidecar
- PostgreSQL Database

Docker shall be used to provide environment parity across development, testing, and production environments while isolating runtime dependencies for the Java and Python components. This approach minimizes platform-specific configuration issues and ensures consistent execution across different operating systems.

Acceptance Criteria

- All application components execute successfully within their respective containers.
- Containerized deployment behaves consistently across supported platforms.

Hardware Compatibility

Description

The system shall maintain compatibility with commonly used desktop computing environments.

Requirements

The application shall support installation and execution on hardware capable of running supported operating systems and container platforms where applicable.

The software shall not depend on:

- Proprietary hardware devices.
- Specialized sensors.
- Custom communication peripherals.

- Dedicated AI accelerator hardware.

GPU acceleration shall be optional and used only when available to improve AI inference performance.

Acceptance Criteria

- The application operates successfully on standard desktop and server hardware.
- Optional hardware enhancements improve performance without affecting core functionality.

3.9.3 Software Interfaces

The Clario platform consists of multiple software components that interact through well-defined interfaces to provide secure, reliable, and efficient customer support services. The software follows a **Hybrid Monolithic Architecture**, where the transactional application and user interface are managed by the Spring Boot application, while AI orchestration, Retrieval-Augmented Generation (RAG), and machine learning functionalities are implemented within a dedicated Python FastAPI ML Sidecar. These components communicate through internal REST APIs over the local network, ensuring logical separation while maintaining low communication overhead.

All software interfaces shall utilize standardized communication protocols, structured data formats, and secure authentication mechanisms to ensure interoperability, maintainability, and extensibility.

User Interface to Backend Interface

Description

The graphical user interface shall communicate with the backend application to perform all business operations.

Connected Components

- Web application using React (Next.js)
- React Frontend
- Spring Boot Backend

Communication Protocol

- HTTP / HTTPS
- RESTful API

Data Format

- JSON

Primary Functions

- User authentication
- Ticket management
- User profile management
- Dashboard data retrieval
- Knowledge base management
- Administrative operations
- Notification retrieval
- System configuration

Acceptance Criteria

- The frontend communicates exclusively through documented REST APIs.
- All requests and responses conform to the published API specification.

Spring Boot to Python ML Sidecar Interface

Description

The Spring Boot application shall communicate with the Python FastAPI ML Sidecar to execute AI-assisted processing tasks. This interface represents the primary integration point between the transactional system and the AI subsystem. The Spring Boot application coordinates business operations while delegating AI-specific functionality—including ticket classification,

Retrieval-Augmented Generation (RAG), LangGraph orchestration, and response generation—to the Python service.

Connected Components

- Spring Boot Backend
- Python FastAPI ML Sidecar

Communication Protocol

- HTTP
- REST API
- Localhost Communication (Development)
- Docker Internal Network (Deployment)

Data Format

- JSON

Primary Functions

- Ticket processing requests
- AI workflow execution
- Ticket classification
- Response generation
- Confidence scoring
- Workflow status retrieval
- AI validation requests

Acceptance Criteria

- Spring Boot successfully invokes FastAPI endpoints.
- AI processing requests return structured responses.
- Communication failures are detected and handled gracefully.

Backend to PostgreSQL Database Interface

Description

The Spring Boot application shall interact with PostgreSQL for transactional data management.

Connected Components

- Spring Boot Backend
- PostgreSQL Database

Interface Technology

- JDBC
- Spring Data JPA / Hibernate ORM

Primary Functions

- User management
- Authentication data
- Customer tickets
- Workflow state
- Configuration data
- Audit logs
- System settings

Acceptance Criteria

- Database transactions maintain ACID properties.
- Database operations preserve referential integrity.
- Connection pooling is managed efficiently.

Python ML Sidecar to ChromaDB Interface

Description

The Python FastAPI ML Sidecar shall communicate with ChromaDB to perform vector storage and semantic retrieval operations.

Connected Components

- Python FastAPI ML Sidecar
- ChromaDB

Interface Technology

- ChromaDB Python Client

Primary Functions

- Store vector embeddings
- Semantic similarity search
- Knowledge retrieval
- Embedding metadata management
- Document indexing

Acceptance Criteria

- Knowledge retrieval returns relevant documents.
- Vector indexes remain synchronized with the knowledge base.
- Retrieval performance satisfies system requirements.

AI Provider Interface

Description

The AI service layer shall communicate with supported Large Language Model (LLM) providers through standardized APIs.

Connected Components

- Python FastAPI ML Sidecar
- External AI Providers

Supported Providers

- OpenAI
- Google Gemini
- Ollama (Local Deployment)
- Other compatible LLM providers

Communication Protocol

- HTTPS
- REST API

Primary Functions

- Prompt submission
- Response generation
- Embedding generation
- Model inference

Acceptance Criteria

- AI providers can be configured without modifying business logic.
- AI responses are successfully returned through the service abstraction layer.
- Provider failures are handled through fallback or escalation mechanisms.

LangGraph Agent Interface

Description

The AI orchestration subsystem shall coordinate specialized AI agents using LangGraph.

Connected Components

- Supervisor Agent
- Classification Agent
- Retrieval Agent
- Response Generation Agent
- Validation Agent
- Escalation Agent

Primary Functions

- Workflow orchestration
- Agent coordination
- State management
- Decision routing
- Workflow execution

Acceptance Criteria

- Agents execute according to the defined workflow.
- Workflow state is preserved throughout execution.
- Agent interactions remain traceable.

Authentication Interface

Description

The authentication subsystem shall provide secure identity verification and session management.

Connected Components

- Spring Boot Backend
- Authentication Service
- User Database

Interface Technology

- JWT Authentication
- Refresh Tokens
- Role-Based Access Control (RBAC)

Primary Functions

- User authentication
- Token generation
- Token validation
- Session management
- Permission verification

Acceptance Criteria

- Authenticated users receive valid access tokens.
- Unauthorized requests are rejected.
- Authentication events are logged.

Knowledge Base Interface

Description

The Knowledge Base Management subsystem shall provide interfaces for maintaining organizational knowledge used by the RAG pipeline.

Connected Components

- Spring Boot Backend
- Python FastAPI ML Sidecar
- ChromaDB
- Document Repository

Primary Functions

- Document upload
- Document indexing
- Metadata management
- Knowledge retrieval
- Document version management

Acceptance Criteria

- Uploaded documents are successfully indexed.
- Updated documents become available for retrieval after processing.
- Document metadata remains synchronized.

Logging and Monitoring Interface

Description

The platform shall provide standardized interfaces for operational logging and system monitoring.

Connected Components

- Spring Boot Backend
- Python FastAPI ML Sidecar
- Logging Service
- Monitoring Service

Primary Functions

- Application logging
- Audit logging
- AI workflow logging
- Performance monitoring
- Health monitoring

Acceptance Criteria

- Operational events are recorded consistently.
- Logs support end-to-end request tracing.
- Monitoring information is available to administrators.

Notification Interface

Description

The notification subsystem shall provide interfaces for delivering operational and workflow notifications.

Connected Components

- Spring Boot Backend
- Notification Service
- User Interface

Primary Functions

- Workflow notifications
- Ticket status updates
- Administrative alerts
- System notifications

Acceptance Criteria

- Notifications are delivered to authorized users.
- Notification status is tracked and recorded.

External Integration Interface

Description

The platform shall support future integration with external enterprise systems through standardized service interfaces.

As identified in the system architecture, the Spring Boot application may serve as an enterprise integration gateway, enabling communication with external systems such as Learning Management Systems (LMS), customer relationship management platforms, or other organizational services without impacting the AI processing subsystem.

Supported Integration Types

- RESTful Web Services
- SOAP Services (where required)
- Third-party APIs
- Enterprise Information Systems

Primary Functions

- Data exchange
- Authentication with external systems
- Service orchestration
- Request transformation
- Response processing

Acceptance Criteria

- External services integrate without affecting core application functionality.
- Communication failures are detected and logged.

Software Interface Design Considerations

All software interfaces shall comply with the following design principles:

- Interfaces shall follow the principles of loose coupling and high cohesion.
- All service contracts shall be documented using the OpenAPI Specification where applicable.
- Data exchanged between services shall use JSON as the standard message format.
- Authentication and authorization shall be enforced for protected interfaces.
- Internal service failures shall not propagate uncontrolled exceptions to client applications.
- Interface versioning shall support backward compatibility wherever practical.
- Service communication shall be logged to support monitoring, auditing, and troubleshooting.
- The separation between the transactional Spring Boot application and the Python ML Sidecar shall be maintained to preserve modularity, facilitate independent maintenance, and enable future migration toward a microservices architecture if enterprise-scale requirements emerge.

3.9.4 Communications Interfaces

The Clario platform shall utilize secure, reliable, and standardized communication mechanisms to enable data exchange between its internal components and external services. The communication architecture supports interactions between the Web application, Spring Boot backend, Python FastAPI ML Sidecar, PostgreSQL database, ChromaDB vector database, and external AI service providers. The system shall employ industry-standard communication protocols, structured data formats, and secure transmission mechanisms to ensure interoperability, maintainability, and data integrity.

The communication interfaces shall support both synchronous and asynchronous interactions where appropriate while maintaining secure and efficient information exchange across all software components. The communication architecture

follows the project's Hybrid Monolithic Architecture, where the Spring Boot application and the Python ML Sidecar communicate through internal REST APIs over the local network or Docker bridge network.

Client–Server Communication Interface

Description

The desktop application shall communicate with the backend services through secure RESTful web services.

Connected Components

- Web application using React (Next.js)
- React Frontend
- Spring Boot Backend

Communication Protocol

- HTTP (Development)
- HTTPS (Production)

Communication Method

- RESTful Request–Response

Data Format

- JSON (UTF-8)

Functions

- User authentication
- Ticket management
- Dashboard data retrieval
- Knowledge base management
- Administrative operations
- Notification retrieval
- AI workflow requests

Acceptance Criteria

- All client requests are transmitted using the supported communication protocol.
- Responses are returned in valid JSON format.
- Communication failures are detected and reported to the user.

Internal Service Communication

Description

The Spring Boot backend shall communicate with the Python FastAPI ML Sidecar to execute AI-assisted operations.

Connected Components

- Spring Boot Backend
- Python FastAPI ML Sidecar

Communication Protocol

- HTTP REST API

Network

- Localhost Interface (Development)
- Docker Internal Bridge Network (Production)

Data Format

- JSON

Functions

- Ticket classification
- AI workflow execution

- Retrieval-Augmented Generation (RAG)
- Response generation
- Confidence evaluation
- Workflow status retrieval

This communication channel enables the transactional backend to delegate AI-specific operations to the FastAPI service while maintaining clear separation between business logic and AI processing.

Acceptance Criteria

- Internal service communication completes successfully under normal operating conditions.
- Service failures are detected through timeout and health monitoring mechanisms.
- Communication errors are logged for troubleshooting.

Database Communication Interface

Description

The backend services shall communicate with persistent storage using native database communication protocols.

Connected Components

- Spring Boot Backend ↔ PostgreSQL
- Python FastAPI ML Sidecar ↔ ChromaDB

Communication Technologies

- PostgreSQL JDBC Driver
- ChromaDB Python Client API

Functions

- Transaction processing
- User management
- Ticket persistence
- Knowledge retrieval
- Vector storage
- Metadata management

Acceptance Criteria

- Database communication maintains data integrity.
- Database connections are managed through connection pooling where applicable.
- Communication failures are handled without data corruption.

External AI Service Communication

Description

The Python FastAPI ML Sidecar shall communicate with supported external Large Language Model (LLM) providers for AI inference and embedding generation.

Supported Providers

- OpenAI
- Google Gemini
- Ollama (Local Deployment)
- Other supported LLM providers

Communication Protocol

- HTTPS

Communication Method

- RESTful APIs
- Streaming APIs (where supported)

Data Format

- JSON

Functions

- Prompt submission
- AI inference
- Embedding generation
- Response retrieval

Acceptance Criteria

- AI requests are securely transmitted.
- Provider responses are validated before processing.
- Communication failures trigger appropriate retry or fallback mechanisms.

Authentication Communication Interface

Description

Authentication and authorization information shall be securely exchanged between the client application and backend services.

Communication Protocol

- HTTPS

Authentication Mechanism

- JSON Web Tokens (JWT)
- Refresh Tokens

Functions

- User login
- Token validation
- Session renewal
- Authorization verification

Acceptance Criteria

- Authentication credentials are transmitted securely.
- Expired or invalid tokens are rejected.
- Session renewal occurs without exposing sensitive information.

Knowledge Base Communication Interface

Description

The system shall support communication between the document repository, indexing services, and the vector database for Retrieval-Augmented Generation (RAG).

Connected Components

- Spring Boot Backend
- Python FastAPI ML Sidecar
- ChromaDB
- Document Storage

Communication Method

- Internal REST API
- Local File System Access (for document ingestion)

Functions

- Document upload
- Document indexing

- Embedding generation
- Semantic retrieval
- Metadata synchronization

Acceptance Criteria

- Uploaded documents are successfully processed.
- Knowledge indexes remain synchronized with stored documents.
- Retrieval requests return indexed content.

Monitoring and Logging Communication Interface

Description

The platform shall communicate operational and diagnostic information between application components and monitoring services.

Connected Components

- Spring Boot Backend
- Python FastAPI ML Sidecar
- Logging Service
- Monitoring Dashboard

Communication Method

- Structured Log Events
- Health Check Endpoints

Functions

- Health monitoring
- Performance monitoring
- Error reporting
- Audit logging
- AI workflow monitoring

Acceptance Criteria

- Operational events are communicated successfully.
- Monitoring systems receive real-time service status information.
- Critical failures generate alerts.

Network Communication Requirements

Description

The system shall operate over standard TCP/IP networks and support both local and Internet-based communication.

Requirements

- All application communication shall use TCP/IP networking.
- External services shall be accessed through secure Internet connections.
- Internal services shall communicate over trusted local networks.
- Communication timeouts shall be configurable.
- Retry mechanisms shall be implemented for transient communication failures.

Acceptance Criteria

- Network communication remains stable under normal operating conditions.
- Temporary network failures are handled gracefully.

Communication Security Requirements

Description

All communication channels shall implement appropriate security measures to protect system data and user information.

Requirements

- HTTPS shall be used for all external communications.
- Sensitive information shall be encrypted during transmission.
- Authentication tokens shall never be transmitted in plain text outside secure channels.
- API keys and credentials shall be securely managed.
- Communication logs shall exclude confidential information.

Acceptance Criteria

- Sensitive information remains protected during transmission.
- Secure communication protocols are enforced throughout the application.

Error Handling and Recovery

Description

Communication interfaces shall detect failures and provide mechanisms for graceful recovery.

Requirements

The system shall:

- Detect communication timeouts.
- Retry transient failures where appropriate.
- Log communication errors.
- Return standardized error responses.
- Notify users of communication failures without exposing internal implementation details.

Acceptance Criteria

- Communication failures do not cause application crashes.
- Error responses follow the documented API specification.
- Failed communications are recorded for diagnostics.

Communication Performance Requirements

Description

Communication interfaces shall provide efficient data exchange while minimizing latency.

Requirements

- Internal communication between the Spring Boot application and the Python ML Sidecar shall introduce minimal overhead.
- API responses shall minimize unnecessary data transfer.
- Frequently accessed information shall be optimized through efficient request handling.
- Long-running AI operations shall support asynchronous processing where appropriate.

Acceptance Criteria

- Internal service communication meets the performance targets defined in Section 3.5.
- Network utilization remains efficient under expected workloads.

Communication Design Considerations

The communication architecture shall adhere to the following principles:

- Standardized RESTful APIs shall be used for service communication.
- JSON shall be the standard data interchange format.
- Interfaces shall remain loosely coupled to facilitate future scalability.
- Communication protocols shall be platform independent.
- All service endpoints shall be documented using the OpenAPI Specification.

- Internal communication between the Spring Boot application and the Python FastAPI ML Sidecar shall preserve the architectural separation between transactional processing and AI services while supporting future migration toward a distributed microservices architecture if required.

3.10 Database Requirements

The system requires a relational database (PostgreSQL 16+) for maintaining transactional integrity of customer tickets, user accounts, and system logs using ACID properties. Additionally, a vector database (ChromaDB) is required to store and index high-dimensional semantic embeddings (generated via all-MiniLM-L6-v2) to support the Retrieval-Augmented Generation (RAG) subsystem. Both databases must support containerized deployment via Docker volumes.

3.11 Licensing, Legal, Copyright, and Other Notices

The Clario platform utilizes open-source libraries that are governed by their respective permissive licenses (e.g., MIT, Apache 2.0). Any fine-tuned models derived from open-weight base models (such as Mistral or Llama) are subject to the acceptable use policies of their original creators. User data processed through external LLM providers must adhere to the data privacy and retention agreements specified by those providers.

3.12 Applicable Standards

- **IEEE 29148:2018:** Systems and software engineering — Life cycle processes — Requirements engineering.
- **REST Architectural Style:** For internal service-to-service communication.
- **OpenAPI Specification (OAS 3.0):** For API contract definitions between the Spring Boot gateway and the Python ML sidecar.
- **JSON Web Token (JWT) Standard (RFC 7519):** For stateless authentication.

4. Supporting Information

4.1 Architecture References

Please refer to the Clario Software Architecture Document (SAD) for detailed component diagrams, sequence flows, and exact deployment port mappings governing the Hybrid Monolithic structure.

4.2 Academic References

- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020).
- QLoRA: Efficient Finetuning of Quantized LLMs (Detrmers et al., 2023).
- LangGraph: Multi-Agent State Workflows (LangChain, 2024).